SH

LINUX, SHELL & GIT

Linux, Shell & Git

> రోజువారీ పనిముట్లు – permissions, processes, pipes, మరియు Git యొక్క నిజమైన నమూనా (commits ఒక graph).

CS fundamentals – self-taught / non-CS background నుంచి వచ్చినవారికి SSE interview lo అడిగే లోతు వరకు. ప్రతి concept ని MERN / JavaScript ప్రపంచంతో ముడిపెట్టి.

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 1 – Linux (ప్రతి engineer కి తప్పనిసరి పునాది)

1. Linux అంటే ఏమిటి, ఎందుకు (kernel vs OS, distros, servers ఎందుకు Linux వాడతాయి, terminal/shell/bash)
2. Filesystem Hierarchy (`/ /home /etc /var /bin /usr`, absolute vs relative paths, `~ . ..`)
3. Navigation & File Operations (`ls/cd/pwd`, `cp/mv/rm/mkdir/touch/ln`, `cat/less/head/tail/nano`)
4. Permissions (rwx, `chmod` numeric+symbolic, `chown`, users/groups, `sudo`/root)
5. Processes (`ps/top/htop`, `kill`/signals, `jobs/fg/bg`, `&`/ `nohup`, `systemctl` services)
6. Text Processing Power Tools (pipes `|`, redirection `> >> <`, `grep/egrep`, `sed`, `awk`, `find`, `wc/sort/uniq/cut/xargs`)
7. Networking & Remote (`curl/wget`, `ping`, `ss/netstat`, `ssh`, `scp/rsync`)
8. Environment & Packages (env vars, `PATH`, `export`, `.bashrc/.zshrc`, `apt/yum/brew`, `which/man`)

Part 2 – Shell Scripting (automation)

9. Shell Scripting (shebang, variables, quoting, `if/case`, `for/while` loops, functions, arguments `$1/$@`, exit codes, ఒక real useful script)

Part 3 – Git (version control – team లో survive అవ్వడానికి)

10. Git అంటే ఏమిటి, ఎందుకు (version control ఎందుకు, git vs GitHub, git snapshots ఎలా store చేస్తుంది – 3 areas: working/staging/repo)
11. Git Basics (`init/clone`, `add/commit`, `status/log/diff`, `.gitignore`)
12. Branching & Merging (`branch/switch/checkout`, `merge`, fast-forward vs 3-way, `HEAD`)
13. Rebase vs Merge + Conflicts resolve చేయడం (step-by-step conflict resolution)
14. Remotes & Collaboration (`remote/origin`, `push/pull/fetch`, tracking branches, PR workflow, fork/upstream)
15. Undoing Mistakes (`reset` soft/mixed/hard, `revert`, `restore`, `stash`, `amend`, `reflog`, `cherry-pick`)
16. Git Workflows (feature-branch, gitflow, trunk-based) + best practices

Part 4 – Reference (interview రోజు)

17. Interview Q&A + Command Cheat-sheet + Common Mistakes

Part 1 — Linux

> ప్రతి production server — నీ MERN app run అయ్యే EC2, DigitalOcean droplet, Docker container — దాదాపు అన్నీ **Linux**. React build local లో అవుతుంది, కానీ అది deploy అయ్యేది Linux మీద. Node process crash అయితే logs చూడాలి, port block అయితే process kill చేయాలి, disk full అయితే పెద్ద files వెతకాలి — ఇవన్నీ **terminal** లో commands తో. ఈ Part లో Linux ని absolute basics నుండి — file system, permissions, processes, text tools, networking — practical గా నేర్చుకుంటాం. ఇది వస్తే, ఏ server మీదైనా నువ్వు కంగారుపడవు.

---

1. Linux అంటే ఏమిటి, ఎందుకు

వివరణ

Linux ఒక operating system (OS) — Windows, macOS లాగా. కానీ చాలా మంది "Linux" అని అనేది నిజానికి **kernel**. ఈ తేడా అర్థం చేసుకోవడం చాలా ముఖ్యం:

- **Kernel** = OS యొక్క **గుండె** (core). Hardware (CPU, RAM, disk, network card) ని software తో మాట్లాడించే layer. మీ program "ఈ file చదువు" అంటే, kernel disk కి వెళ్ళి data తెచ్చి ఇస్తుంది. Process scheduling, memory management, file system, device drivers, networking — అన్నీ kernel పని. దీన్ని **Linus Torvalds** 1991 లో రాశాడు.
- **OS (distribution/distro)** = kernel + చుట్టూ ఉన్న అన్ని tools (shell, file utilities, package manager, libraries, desktop). Kernel ఒక్కటే వాడలేం — దాని చుట్టూ userland tools కావాలి. ఆ complete package నే **distro** అంటారు.

అంటే: **Linux = kernel; Ubuntu/Debian/CentOS = ఆ kernel మీద build చేసిన complete OS (distro).**

Distros (r.వేర్వేరు flavors):

Distro	Family	ఎక్కడ వాడతారు
Ubuntu	Debian-based	Beginners, cloud servers, dev machines. అత్యంత popular. `apt` package manager.
Debian	Debian	Stable servers. Ubuntu దీని మీదే build అయ్యింది.
CentOS / RHEL / Rocky / Alma	Red Hat	Enterprise servers, corporates. `yum` / `dnf` package manager.
Amazon Linux	RHEL-based	AWS EC2 default. Node deploy చేసే చోటు.
Alpine	Independent	Docker images (చాలా చిన్నది, ~5MB). `node:alpine` image ఇదే.

Servers ఎందుకు దాదాపు అన్నీ Linux?

1. **Free & open-source** – license fee లేదు. 1000 servers కి Windows license కొంటే లక్షలు; Linux ఉచితం.
2. **Stable & lightweight** – GUI లేకుండా (headless) నెలల తరబడి restart లేకుండా run అవుతుంది. Resources RAM/CPU app కి మిగులుతాయి, desktop కి కాదు.
3. **Powerful CLI + automation** – అన్నీ commands/scripts తో automate చేయవచ్చు. DevOps, CI/CD, Docker అన్నీ దీని మీదే.
4. **Security & permissions** – strong user/permission model. Multi-user servers కి perfect.
5. **Ecosystem** – Docker, Kubernetes, nginx, Node, Python – అన్నీ Linux-first.

Terminal vs Shell vs Bash – గందరగోళం clear చేద్దాం:

- **Terminal** = నువ్వు type చేసే **window/app** (iTerm, GNOME Terminal, VS Code terminal). ఇది కేవలం text ని display చేసే, input తీసుకునే డబ్బా. (పాత రోజుల్లో physical "terminal" hardware ఉండేది – ఆ పేరు అలాగే ఉండిపోయింది.)
- **Shell** = terminal లో నువ్వు type చేసిన commands ని **అర్థం చేసుకుని, execute చేసే program**. నీకు, kernel కి మధ్య translator. `ls` type చేస్తే, shell దాన్ని అర్థం చేసుకుని kernel ని అడిగి output చూపిస్తుంది.
- **Bash** = ఒక **specific shell** (Bourne Again SHell). చాలా Linux servers default shell. macOS లో ఇప్పుడు default **zsh** (Bash కి superset లాంటిది). ఇంకా `sh`, `fish` లాంటివి ఉన్నాయి.

అంటే: **Terminal = TV (screen); Shell = remote control (commands interpret); Bash = ఒక particular remote brand.**

Real-life Scenario

> **Linux = ఒక పెద్ద restaurant kitchen.**
>
> - **Kernel** = head chef – actual వంట (hardware access) చేసేవాడు. మీరు నేరుగా అతనితో మాట్లాడలేరు.
> - **Shell** = waiter – మీ order (`ls`, `npm start`) తీసుకుని chef కి చెప్పి, వచ్చిన dish (output) మీకు తెచ్చిస్తాడు. Bash, zsh = వేర్వేరు waiters, కానీ పని ఒకటే.
> - **Terminal** = మీరు కూర్చున్న table – ఇక్కడే order ఇస్తారు, food వస్తుంది.
> - **Distro (Ubuntu)** = మొత్తం restaurant – chef + waiters + menu + furniture. Kitchen ఒక్కటే restaurant కాదు; దాని చుట్టూ అంతా కలిపితేనే restaurant.
>
> MERN developer గా నువ్వు waiter (shell) తో మాట్లాడతావు – "Node app run చెయ్, ఈ log చూపించు, ఈ file copy చెయ్" – waiter chef (kernel) చేత చేయిస్తాడు.

Commands – మొదటి పరిచయం

```
# నేను ఏ OS/kernel మీద ఉన్నాను?
uname -a
# Linux ip-172-31-5-1 5.15.0-1052-aws #57-Ubuntu SMP x86_64 GNU/Linux
```

```
#  ^kernel  ^hostname   ^kernel-version              ^architecture

# ఏ distro? (servers లో చాలా ఉపయోగం)
cat /etc/os-release
# NAME="Ubuntu"
# VERSION="22.04.3 LTS (Jammy Jellyfish)"
# ...

# నేను ఏ shell వాడుతున్నాను?
echo $SHELL
# /bin/bash        (లేదా /bin/zsh macOS లో)

# ఈ system లో ఏ shells available?
cat /etc/shells
# /bin/sh
# /bin/bash
# /bin/zsh

# నేను ఎవరు? (which user)
whoami
# ubuntu

# ఇప్పుడు time & uptime (server ఎంతసేపటి నుండి run అవుతోంది)
uptime
# 14:32:01 up 12 days,  3:45,  1 user,  load average: 0.08, 0.03, 0.01
```

> **గమనిక:** `$SHELL` = నీ *login* shell (default). నువ్వు ప్రస్తుతం run చేస్తున్న shell వేరే కావచ్చు – దాన్ని `echo $0` చూపిస్తుంది.

Kernel ఏం చేస్తుంది – 5 ముఖ్య పనులు

పని	ఏం చేస్తుంది	MERN లో ఉదాహరణ
Process management	ఏ program ఎప్పుడు CPU వాడాలో schedule	Node process, MongoDB process ని manage
Memory management	RAM ని programs కి allocate/free	Node app కి heap memory ఇవ్వడం
File system	Files/directories read/write	`package.json` చదవడం, logs రాయడం
Device drivers	Hardware (disk, network) తో మాట్లాడటం	Network card ద్వారా HTTP request పంపడం
System calls	Programs kernel ని అడిగే API	`open()`, `read()`, `write()`, `fork()`

Key Points

- **Linux = kernel (గుండె, hardware ని manage చేస్తుంది); distro (Ubuntu/CentOS) = kernel + tools = complete usable OS.**
- **Kernel** ని Linus Torvalds రాశాడు; ప్రపంచంలో అత్యధిక servers, Android, cloud దీని మీదే నడుస్తాయి.
- **Terminal = window; Shell = interpreter; Bash = ఒక specific shell.** ఈ మూడూ వేర్వేరు layers.

- Servers Linux ఎందుకు: **free, stable, lightweight, scriptable, secure, ecosystem.**
- MERN deploy: నీ app దాదాపు ఎప్పుడూ Linux server (EC2/Docker) మీద run అవుతుంది – అందుకే Linux CLI తప్పనిసరి.

Interview దృష్టి

Q: Linux మరియు Unix తేడా ఏమిటి? A: Unix ఒక పాత proprietary OS (1970s, AT&T Bell Labs). Linux అనేది Unix *లాంటి* (Unix-like) kernel, కానీ Unix code వాడకుండా scratch నుండి రాశారు (open-source). macOS కూడా Unix-based (BSD). అందుకే macOS terminal commands చాలావరకు Linux లాగే పనిచేస్తాయి – POSIX standard follow చేస్తాయి.

Q: Terminal, shell, bash ఒకటేనా? A: కాదు. Terminal = text input/output చేసే program (UI). Shell = ఆ input ని interpret చేసి execute చేసే program. Bash = ఒక particular shell implementation. ఒక terminal లో వేర్వేరు shells (bash, zsh) run చేయవచ్చు.

Q: Server కి GUI ఎందుకు ఉండదు? A: GUI RAM/CPU వాడుతుంది, security surface పెంచుతుంది, remote access కష్టం చేస్తుంది. Servers headless (CLI-only) – resources app కి, automation scripts తో manage. అందుకే SSH + CLI నేర్చుకోవడం mandatory.

---

2. Filesystem Hierarchy – Linux file system ఎలా organized

LINUX FILESYSTEM · ఏది ఎక్కడ ఉంటుంది

FILESYSTEM HIERARCHY – ఏది ఎక్కడ

`/bin, /usr/bin`	executables – ls, grep, node
`/etc`	config files
`/var`	మారే data – logs, cache
`/home`	user files
`/tmp`	తాత్కాలికం – reboot కి పోతుంది
`/proc`	kernel యొక్క virtual view

/proc ఒక నిజమైన ఫైల్‌సిస్టం కాదు

అది kernel memory యొక్క ఒక దృశ్యం.

`cat /proc/cpuinfo`, `/proc/[pid]/status`.Disk మీద ఏమీ లేదు – చదివినప్పుడు kernel ఆ క్షణం సమాధానం తయారు చేస్తుంది.

వివరణ

Windows లో `C:\`, `D:\` లాంటి drives ఉంటాయి. **Linux లో అలా కాదు** – అంతా ఒక్క **root** (`/`) నుండి మొదలయ్యే **ఒకే tree**. ప్రతి disk, USB, network drive కూడా ఈ tree లో ఒక folder లాగా *mount* అవుతుంది. అంటే Linux లో "everything is a file" మరియు "అన్నీ `/` కింద".

`/` (root – గమనిక: ఇది root *user* కాదు, root *directory*) tree యొక్క పైమెట్టు. దాని కింద standard folders ఉంటాయి – ఈ layout ని **FHS (Filesystem Hierarchy Standard)** అంటారు, అన్ని distros దీన్ని follow చేస్తాయి. అందుకే ఏ Linux server లోనైనా `/etc`, `/var` ఎక్కడ ఉంటాయో తెలిసిపోతుంది.

ముఖ్య directories – ఒక్కొక్కటి ఏం కోసం:

Directory	పూర్తి పేరు	ఏం ఉంటుంది	MERN లో ఎప్పుడు తగుల్తుంది
`/`	root	tree మొదలు; అన్నిటికీ మూలం	–
`/home`	home	ప్రతి normal user కి personal folder (`/home/ubuntu`)	నీ app code ఇక్కడే క్లోన్ చేస్తావు
`/root`	root's home	`root` user యొక్క home (గమనిక: `/home` లో కాదు)	root గా login అయితే
`/etc`	et cetera (config)	system-wide **configuration** files	`nginx.conf`, `/etc/hosts`, ssh config
`/var`	variable	మారుతూ ఉండే data – **logs**, caches, spool	`/var/log/nginx/`, app logs
`/bin` `/usr/bin`	binaries	executable **commands** (`ls`, `cat`, `node`)	`which node` → `/usr/bin/node`
`/usr`	Unix system resources	user programs, libraries, docs	installed software ఇక్కడ
`/tmp`	temporary	temp files (reboot కి delete అవుతాయి)	build temp files
`/opt`	optional	third-party software (manual installs)	కొన్ని apps
`/dev`	devices	hardware devices as files (`/dev/null`, disks)	`/dev/null` కి output పంపడం
`/proc`	process	running processes, kernel info (virtual)	`/proc/cpuinfo`, PID info
`/mnt` `/media`	mount	external drives/USB mount అయ్యే చోటు	volumes attach

> **గుర్తుంచుకో:** config కావాలంటే `/etc`, logs కావాలంటే `/var/log`, commands ఎక్కడ ఉన్నాయో అంటే `/usr/bin`. ఈ మూడు SSE గా రోజూ తగుల్తాయి.

Real-life Scenario

> **Linux file system = ఒక పెద్ద office building.**
>
> - `/` = building యొక్క main entrance (ground). అన్ని floors ఇక్కడి నుండే.
> - `/home` = employees personal cabins (ప్రతి ఒక్కరికి ఒకటి – `/home/ubuntu`, `/home/john`).
> - `/etc` = building manager office – అన్ని rules, settings (WiFi password, AC config) ఇక్కడ.
> - `/var/log` = security camera recordings room – ఏం జరిగిందో అంతా (logs) ఇక్కడ record.
> - `/bin`, `/usr/bin` = tools store room – hammer, screwdriver (commands) ఇక్కడ.
> - `/tmp` = whiteboard – రాసుకుంటాం, రోజు చివర తుడిచేస్తారు (reboot కి clear).
>
> ఏదైనా వెతకాలంటే ఏ room కి వెళ్ళాలో తెలిస్తే, పని సులభం. Server issue వచ్చినప్పుడు, "logs ఏ room లో?" అంటే నేరుగా `/var/log`.

Absolute vs Relative paths – చాలా ముఖ్యం

ఒక file ని రెండు రకాలుగా చెప్పవచ్చు:

- **Absolute path** = ఎప్పుడూ `/` (root) నుండి మొదలవుతుంది. ఎక్కడ నుండి type చేసినా అదే file. ఉదా: `/home/ubuntu/app/server.js`. (GPS full address లాంటిది.)
- **Relative path** = **నువ్వు ప్రస్తుతం ఉన్న folder** (current directory) నుండి. ఉదా: `app/server.js` లేదా `../config`. (ఇక్కడి నుండి "రెండు ఇళ్ళు ముందుకు" లాంటిది.)

Special symbols – వీటిని రోజూ వాడతావు:

Symbol	అర్థం	ఉదాహరణ
`/`	root directory (path మొదట్లో) OR separator	`/etc/nginx`
`~`	ప్రస్తుత user యొక్క home (`/home/ubuntu`)	`cd ~`, `~/app`
`.`	**ప్రస్తుత** directory (current)	`./script.sh`, `node .`
`..`	**parent** directory (ఒక level పైకి)	`cd ..`, `../config.js`
`-`	ముందు ఉన్న directory (previous)	`cd -` (back and forth)

Commands

```
# నేను ఇప్పుడు ఎక్కడ ఉన్నాను? (Print Working Directory)
pwd
# /home/ubuntu/my-mern-app

# home కి వెళ్ళు (~ = home shortcut)
cd ~
pwd
```

```
# /home/ubuntu

# absolute path తో — ఎక్కడ నుండైనా పనిచేస్తుంది
cd /var/log
pwd
# /var/log

# relative path తో — ఇక్కడి నుండి ఒక level పైకి, తర్వాత లోపలికి
cd ..            # ఇప్పుడు /var లో ఉన్నాం
cd ./log         # ./ = ఇక్కడి నుండి; తిరిగి /var/log
pwd
# /var/log

# రెండు folders మధ్య toggle — cd - ముందు folder కి తిరిగి
cd ~/my-mern-app
cd /etc/nginx
cd -             # తిరిగి ~/my-mern-app కి!
# /home/ubuntu/my-mern-app

# home path ని program లో — MERN లో logs రాయడానికి
echo ~
# /home/ubuntu
echo $HOME       # అదే విలువ, env var గా
# /home/ubuntu
```

Absolute vs Relative — ఎప్పుడు ఏది?

అంశం	Absolute path	Relative path
మొదలు	ఎప్పుడూ `/` నుండి	current directory నుండి
ఎక్కడ నుండైనా	✅ ఎప్పుడూ ఒకటే file	❌ current dir మారితే మారుతుంది
పొడవు	పొడవుగా ఉంటుంది	పొట్టిగా, convenient
ఎప్పుడు వాడాలి	scripts, cron jobs, config (guaranteed correctness)	terminal లో quick navigation
ఉదాహరణ	`/home/ubuntu/app/index.js`	`./index.js`, `../lib`

> **Gotcha (SSE లో చాలా common bug):** Cron job లేదా systemd service లో **relative path** వాడితే fail అవుతుంది — ఎందుకంటే అవి వేరే directory నుండి run అవుతాయి. Scripts లో ఎప్పుడూ **absolute paths** వాడు, లేదా script మొదట్లో `cd` చేసి directory fix చేయి.

Key Points

- Linux file system = ఒకే tree, `/` (root) నుండి. Windows లాగా `C:` `D:` లేవు — అన్నీ `/` కింద mount.
- **FHS standard** వల్ల అన్ని distros లో `/etc` (config), `/var/log` (logs), `/usr/bin` (commands) ఒకే చోట.
- **Absolute** = `/` నుండి (ఎక్కడ నుండైనా same); **Relative** = current dir నుండి. Scripts/cron లో absolute వాడు.
- `~` = home, `.` = current, `..` = parent, `-` = previous dir. ఇవి రోజూ.

- SSE reflex: config → `/etc`, logs → `/var/log`, "command ఎక్కడ?" → `which <cmd>`.

Interview దృష్టి

Q: `/bin` మరియు `/usr/bin` తేడా? A: చారిత్రకంగా `/bin` = boot/single-user mode లో essential commands (`ls`, `cp`), `/usr/bin` = మిగతా అన్ని user commands. ఆధునిక distros లో `/bin` అనేది `/usr/bin` కి symlink (merged). ఆచరణలో commands రెండింటిలో ఏదో ఒక చోట; `which <cmd>` చెప్తుంది.

Q: `.` (dot) ఎందుకు path లో వాడతాం, `node .` అంటే? A: `.` = current directory. `node .` అంటే "current directory లో main file (package.json లోని `main`) run చెయ్". `./script.sh` అంటే "ఇక్కడి script run చెయ్" – ఎందుకంటే security కోసం current directory PATH లో ఉండదు, అందుకే `./` explicit గా చెప్పాలి.

Q: Server లో disk full అయింది – పెద్ద files ఎక్కడ వెతుకుతావు? A: సాధారణంగా `/var/log` (పెరిగిపోయిన logs) మొదట. `du -sh /var/* | sort -h` తో ఏ folder పెద్దదో చూస్తా. చాలాసార్లు culprit rotate అవ్వని application logs లేదా Docker images (`/var/lib/docker`).

---

3. Navigation & File Operations – రోజువారీ bread-and-butter commands

వివరణ

Terminal లో నీ 80% time ఈ commands మీదే గడుస్తుంది: ఎక్కడ ఉన్నానో చూడటం (`ls`, `pwd`), తిరగడం (`cd`), files create/copy/move/delete చేయడం, content చూడటం. ఇవి "muscle memory" అవ్వాలి – ఆలోచించకుండా వేళ్ళు type చేయాలి. అందుకే ఒక్కొక్కటి flags తో సహా చూద్దాం.

Real-life Scenario

> **Terminal navigation = ఒక పెద్ద library లో తిరగడం.**
>
> - `pwd` = "నేను ఇప్పుడు ఏ floor, ఏ section లో ఉన్నాను?" – board చూడటం.
> - `ls` = ఈ section లో ఏ books (files) ఉన్నాయో చూడటం.
> - `cd` = వేరే section కి నడవడం.
> - `cp/mv/rm` = book ని photocopy / మరో shelf కి తరలించడం / తీసి పారేయడం.
> - `cat/less` = book తెరిచి చదవడం.
>
> Library లో ఎక్కడ ఉన్నావో తెలియకుండా book వెతకలేవు – అలాగే `pwd` / `ls` లేకుండా terminal లో పని చేయలేవు. అందుకే మొదట ఇవి పక్కాగా.

Navigation: `pwd`, `ls`, `cd`

```
pwd                          # ఎక్కడ ఉన్నానో – /home/ubuntu/app
```

```
ls                      # ఈ folder లో files/folders list
# node_modules  package.json  server.js  src

ls -l                   # long format: permissions, owner, size, date
# -rw-r--r-- 1 ubuntu ubuntu  512 Jul 17 10:00 package.json
# drwxr-xr-x 8 ubuntu ubuntu 4096 Jul 17 09:55 src

ls -a                   # hidden files కూడా (. తో మొదలయ్యేవి — .git, .env)
# .  ..  .env  .git  .gitignore  package.json  server.js

ls -la                  # long + hidden కలిపి (అత్యంత common)
ls -lh                  # -h = human-readable sizes (512K, 3.2M — bytes కాదు)
ls -lt                  # time ప్రకారం sort (newest first) — ఏది ఇప్పుడే మారింది?
ls -ltr                 # -r = reverse → newest చివర్లో (logs కి perfect)
ls -R                   # recursive — subfolders అన్నీ కూడా
ls src/                 # వేరే folder లోది చూడటం (అక్కడికి వెళ్ళకుండా)
```

`ls -l` **output ని ఎలా చదవాలి** (SSE గా తెలియాలి):

```
-rw-r--r--   1  ubuntu  ubuntu   512   Jul 17 10:00   package.json
[1]         [2] [3]     [4]      [5]   [6]             [7]
```

1. **type + permissions** (`-` file, `d` directory, `l` symlink; తర్వాత rwx — Topic 4).
2. **link count** (hard links).
3. **owner** (user).
4. **group**.
5. **size** (bytes; `-h` తో human-readable).
6. **last modified** date/time.
7. **name**.

```
cd src                  # src లోకి (relative)
cd /var/log             # absolute
cd ..                   # ఒక level పైకి
cd ~                    # home కి (cd ఒక్కటే కూడా home కి వెళ్తుంది)
cd                      # ~ కి equivalent
cd -                    # ముందు ఉన్న directory కి
```

File & Directory creation: `mkdir`, `touch`

```
mkdir logs              # ఒక folder create
mkdir -p src/api/v1     # -p = parents కూడా create (src, api లేకపోతే కూడా). చాలా useful!
# -p లేకపోతే: "mkdir: cannot create 'src/api/v1': No such file or directory"

touch server.js         # ఖాళీ file create (ఉంటే timestamp update)
touch a.js b.js c.js    # ఒకేసారి multiple files
```

> `touch` **twist:** file ఇప్పటికే ఉంటే content ముట్టకుండా **modified time** ని ఇప్పటికి update చేస్తుంది. Build systems లో "ఈ file మారింది అనుకో" అని force చేయడానికి వాడతారు.

Copy, Move/Rename, Delete: `cp`, `mv`, `rm`

```
# --- cp: copy ---
cp server.js server.backup.js       # file copy
cp -r src src-backup                # -r = recursive (folder copy కి తప్పనిసరి)
cp -i old.js new.js                 # -i = interactive (overwrite ముందు అడుగు)
cp -v *.js dist/                    # -v = verbose (ఏం copy అయిందో చూపు)
cp .env.example .env                # config template నుండి actual config

# --- mv: move OR rename (రెండూ ఒకే command!) ---
mv server.js app.js                 # rename (అదే folder → పేరు మారుతుంది)
mv app.js src/                      # move (వేరే folder కి)
mv src/app.js ./app.js              # move back
mv -i a.js b.js                     # -i = overwrite ముందు అడుగు

# --- rm: remove (⚠️ Recycle bin లేదు — permanent!) ---
rm old.js                           # file delete
rm -r old-folder                    # -r = folder + లోపలి అన్నీ
rm -f locked.js                     # -f = force (అడగకుండా, error ignore)
rm -rf node_modules                 # recursive + force (node_modules తుడవడానికి
classic)
rm -i *.log                         # -i = ప్రతి file కి confirm అడుగు (safe)
```

⚠️ **అత్యంత ప్రమాదకరమైన command:** `rm -rf`. Linux లో **trash/undo లేదు** – delete అంటే గోన్. `rm -rf /` లేదా `rm -rf $VAR/` (VAR ఖాళీ అయితే `/` అవుతుంది!) system ని తుడిచేయగలదు. **Rule:** `rm -rf` ముందు `pwd` చూడు, path double-check చేయి, variables లో trailing content ఉందో లేదో నిర్ధారించు.

Symbolic links: `ln -s`

```
# symlink = ఒక file/folder కి "shortcut/pointer"
ln -s /var/www/app/current /var/www/app/releases/v2.3
#      ^target (అసలు)                  ^link (shortcut)
# ఇప్పుడు 'current' అనేది v2.3 కి point చేస్తుంది.

ls -l /var/www/app/current
# lrwxrwxrwx ... current -> /var/www/app/releases/v2.3
#  ^l = symlink                     ^ ఎటు point చేస్తుందో

# deployment లో classic: 'current' symlink ని కొత్త release కి మార్చడం = instant switch!
ln -sfn /var/www/app/releases/v2.4 /var/www/app/current   # -f force, -n no-deref
```

MERN deployment tip: zero-downtime deploy లో ఇదే pattern – ప్రతి release ని `releases/timestamp` లో పెట్టి, `current` symlink ని కొత్తదానికి flip చేస్తారు. Rollback = symlink వెనక్కి మార్చడం (instant). Capistrano, Deployer ఇలాగే పనిచేస్తాయి.

Viewing content: `cat`, `less`, `head`, `tail`, `nano`

```
cat package.json          # మొత్తం file terminal లో dump (చిన్న files కి)
cat -n server.js          # -n = line numbers తో
```

```
# పెద్ద files కి cat వద్దు — less వాడు (page-by-page, scroll)
less /var/log/nginx/access.log
#   navigation: ↑↓ scroll, Space=next page, /word=search, n=next match, q=quit,
G=end, g=start

head server.js            # మొదటి 10 lines
head -n 20 server.js      # మొదటి 20 lines
tail server.js            # చివరి 10 lines
tail -n 50 error.log      # చివరి 50 lines

# ⭐ SSE లో అత్యంత useful — live logs చూడటం:
tail -f /var/log/app.log  # -f = follow: కొత్త lines వచ్చినట్టు live చూపిస్తుంది (Ctrl+C to
stop)
tail -f app.log | grep ERROR   # live లో ERROR మాత్రమే filter

nano config.txt           # simple terminal editor (Ctrl+O save, Ctrl+X exit)
                          # (vim కూడా ఉంది కానీ beginners కి nano సులభం)
```

File viewers – ఎప్పుడు ఏది:

Command	ఎప్పుడు వాడాలి	గుర్తు
`cat`	చిన్న files, లేదా pipe లో pass చేయడానికి	మొత్తం ఒకేసారి dump
`less`	పెద్ద files scroll/search చేయడానికి	quit = `q`
`head`	file మొదలు చూడటానికి (CSV headers)	top N
`tail`	file చివర, ⭐ **live logs** (`-f`)	bottom N
`nano`	quick edit (config, .env)	save = Ctrl+O

Key Points

- `pwd` (ఎక్కడ), `ls -la` (ఏమున్నాయి, hidden సహా), `cd` (తిరగడం) – muscle memory అవ్వాలి.
- `mkdir -p` = nested folders ఒకేసారి; `touch` = ఖాళీ file / timestamp update.
- `cp -r` (folder copy), `mv` (move **AND** rename ఒకటే command), `rm -rf` (⚠️ permanent, trash లేదు).
- `tail -f` = **live logs** చూడటానికి SSE యొక్క best friend; పెద్ద files కి `less`, `cat` కాదు.
- `ln -s` = symlink (shortcut) – deployment లో `current` release flip చేయడానికి.

Interview దృష్టి

Q: `cp` vs `mv` – internally తేడా? A: `mv` అదే filesystem లో అయితే కేవలం **inode pointer/name** మారుస్తుంది (data copy కాదు) – అందుకే instant, పెద్ద files కి కూడా. `cp` నిజంగా data ని కొత్త చోటికి duplicate చేస్తుంది (slow, రెండు copies). వేరే filesystem కి `mv` అయితే అది copy + delete అవుతుంది.

Q: Accidentally `rm` చేసిన file recover చేయవచ్చా? A: సాధారణంగా కష్టం – trash లేదు. కానీ: (1) file ఇంకా process open చేసి ఉంటే `/proc/<pid>/fd/` నుండి recover చేయవచ్చు, (2) `git` tracked

అయితే `git checkout`, (3) backups/snapshots. అందుకే prevention ముఖ్యం – `rm -i` alias, లేదా `trash-cli` వాడటం.

Q: `tail -f` ఎలా పనిచేస్తుంది? A: File ని open చేసి, end కి seek చేసి, కొత్త data వచ్చిందేమో inode ని poll/inotify తో watch చేస్తుంది; కొత్త bytes వస్తే print చేస్తుంది. Log rotate అయితే `tail -F` (capital) వాడాలి – అది file తిరిగి open చేస్తుంది.

---

4. Permissions – rwx, chmod, chown, sudo

వివరణ

Linux ఒక **multi-user** system – ఒకే server మీద చాలామంది users, చాలా services. అందుకే "ఎవరు ఏ file ని చదవగలరు/రాయగలరు/run చేయగలరు?" అనేది strict గా control చేస్తుంది. ఇదే **permissions** system. Interview లో "ఈ `.sh` script run అవ్వట్లేదు" అంటే 90% సార్లు permission problem (`Permission denied`).

ప్రతి file/folder కి **3 permission types × 3 categories** ఉంటాయి:

3 permission types (ఏం చేయవచ్చు):

Symbol	పేరు	File మీద అర్థం	Directory మీద అర్థం
`r`	read (4)	content చదవడం	లోపల files list చేయడం (`ls`)
`w`	write (2)	content మార్చడం	files create/delete చేయడం
`x`	execute (1)	program గా run చేయడం	లోపలికి `cd` చేయడం

3 categories (ఎవరికి):

Symbol	పేరు	ఎవరు
`u`	user/owner	file owner
`g`	group	file యొక్క group members
`o`	others	మిగతా అందరూ (everyone else)

`ls -l` లో permissions ఎలా చదవాలి

```
-rwxr-xr--
|└┘└┘└┘
| |  |  └── others: r-- (read మాత్రమే)
| |  └───── group:  r-x (read + execute)
| └──────── user:   rwx (read + write + execute)
└────────── type:   - (file), d (dir), l (symlink)
```

అంటే `-rwxr-xr--`: owner అన్నీ చేయవచ్చు, group read+run చేయవచ్చు కానీ మార్చలేరు, others కేవలం read.

LINUX PERMISSIONS · RWX మరియు సంఖ్యలు

rwx – మూడు గుంపులకి మూడేసి అనుమతులు

- rwx r-x r-- → owner: rwx 7 | group: r-x 5 | others: r-- 4 — chmod 754

సంఖ్యలు

r = 4 · w = 2 · x = 1
కలిపితే: rwx = 7, r-x = 5, r-- = 4
chmod 644 = సాధారణ file · 755 = script

Directory మీద x అంటే

చదవడం కాదు – **లోపలికి వెళ్ళడం** (cd).
x లేకపోతే – లోపల ఏముందో తెలిసినా చేరలేరు.

తరచుగా వచ్చే ఉచ్చు

chmod 777 ఎప్పుడూ వాడకండి – "పని చేయడం లేదు" అనే సమస్యకి అది పరిష్కారం కాదు, ప్రమాదం.
సరైన ప్రశ్న: "ఏ user ఈ process ని నడుపుతోంది?" – అది సరిచేస్తే permission సమస్య పోతుంది.

Real-life Scenario

> **Permissions = ఒక office building లో access cards.**
>
> - `r` (read) = room లోకి చూడగలవ (glass door).
> - `w` (write) = room లో వస్తువులు మార్చగలవ.
> - `x` (execute) = room ని "activate" చేయగలవ (machine start).
> - `u/g/o` = నువ్వు owner వా (u), అదే team వా (g), లేక outsider వా (o)?
>
> నీ personal cabin (`~/.ssh/id_rsa` – private key) కి **నీకు మాత్రమే** access (`600`). Team shared folder కి team members కి access (`g`). Public notice board అందరికీ read (`o` కి `r`). SSH private key కి others కి access ఉంటే – SSH "unprotected key" అని refuse చేస్తుంది. అందుకే permissions security కి core.

Numeric (octal) notation – SSE లో ఎక్కువ వాడతారు

ప్రతి permission కి number: **r=4, w=2, x=1**. ఒక category కి వాటిని కూడు:

rwx	లెక్క	Octal
`---`	0	**0**
`--x`	1	**1**
`-w-`	2	**2**
`r--`	4	**4**
`r-x`	4+1	**5**
`rw-`	4+2	**6**
`rwx`	4+2+1	**7**

మూడు categories కి మూడు digits: **u g o**.

Octal	అర్థం	ఎప్పుడు వాడతారు
`755`	rwx r-x r-x	executables, folders, scripts (owner అన్నీ, మిగతా వాళ్ళు run/read)
`644`	rw- r-- r--	normal files (owner edit, మిగతా వాళ్ళు read)
`600`	rw- --- ---	secrets – **SSH keys, .env** (owner మాత్రమే)
`700`	rwx --- ---	private folders/scripts (owner మాత్రమే)
`777`	rwx rwx rwx	⚠️ అందరికీ అన్నీ – **almost ఎప్పుడూ తప్పు** (security hole)

`chmod` – permissions మార్చడం

```
# --- Numeric (absolute — మొత్తం set చేస్తుంది) ---
chmod 755 deploy.sh        # rwx r-x r-x → script run చేయగలిగేలా
chmod 644 index.html       # rw- r-- r-- → normal file
chmod 600 ~/.ssh/id_rsa    # rw- --- --- → private key (తప్పనిసరి!)
chmod -R 755 public/       # -R = recursive (folder లోని అన్నిటికీ)

# --- Symbolic (relative — పెంచడం/తగ్గించడం) ---
chmod +x script.sh         # అందరికీ execute add (script run అవ్వడానికి classic)
chmod u+x script.sh        # owner కి మాత్రమే execute add
chmod g-w file.txt         # group నుండి write తీసేయి
chmod o-rwx secret.txt     # others నుండి అన్నీ తీసేయి
chmod a+r public.txt       # a = all (u+g+o) కి read
chmod u=rw,g=r,o=r file    # explicit set (= 644)
```

> **అత్యంత common use:** shell script రాసి run చేయబోతే `Permission denied` → `chmod +x script.sh` → ఇప్పుడు `./script.sh` పనిచేస్తుంది. ఎందుకంటే script కి execute permission లేకపోతే OS దాన్ని program గా run చేయదు.

`chown` – ownership మార్చడం (owner/group)

```
sudo chown ubuntu server.js             # owner ని ubuntu కి మార్చు
sudo chown ubuntu:www-data app.js       # owner=ubuntu, group=www-data
sudo chown -R ubuntu:ubuntu /var/www/app # recursive (deploy తర్వాత classic)
sudo chgrp www-data file.txt            # group మాత్రమే మార్చు

# MERN scenario: root గా npm install చేసి node_modules root-owned అయ్యింది,
# ఇప్పుడు ubuntu user రాయలేకపోతున్నాడు →
sudo chown -R ubuntu:ubuntu ~/app       # ownership సరిచేయి
```

Users, Groups, sudo, root

- **root** = superuser (UID 0) – అన్నీ permissions bypass చేసే **god mode**. ఏ file నైనా చదవగలడు, delete చేయగలడు. ప్రమాదకరం – తప్పు command system ని పాడు చేస్తుంది.
- **normal user** (ubuntu) = limited. తన home లో మాత్రమే free గా రాయగలడు.

- **sudo** = "**S**uper **U**ser **DO**" – ఒక్క command ని temporarily root గా run చేయడం. Password అడుగుతుంది, logs లో record అవుతుంది.
- **group** = users ని కలిపి manage చేయడానికి. ఉదా: `www-data` group లో web server + deploy user → shared file access.

```
whoami              # ప్రస్తుత user — ubuntu
id                  # user + అన్ని groups
# uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),27(sudo),33(www-data)
groups              # ఏ groups లో ఉన్నానో
sudo apt update     # root privilege అవసరమైన command (password అడుగుతుంది)
sudo -i             # root shell కి switch (prompt # అవుతుంది — జాగ్రత్త)
```

> **Rule of thumb:** ఎప్పుడూ root గా login అయి పని చేయకు. Normal user గా ఉండి, అవసరమైనప్పుడు మాత్రమే `sudo`. Package install (`sudo apt install`), service restart (`sudo systemctl restart nginx`), system config edit (`sudo nano /etc/nginx/...`) – వీటికే sudo. కానీ `npm install` (project లో), git – వీటికి sudo **వద్దు** (ownership పాడవుతుంది).

Key Points

- ప్రతి file: **3 perms (rwx) × 3 categories (u/g/o)**. `ls -l` first column లో కనిపిస్తాయి.
- **Numeric:** r=4, w=2, x=1. `755` = scripts/folders, `644` = files, `600` = secrets (SSH keys, .env).
- `chmod +x script.sh` = script run అవ్వడానికి (అత్యంత common fix). `chmod -R` = recursive.
- `chown user:group` = ownership. `sudo` = temporary root; ఎప్పుడూ root గా stay అవ్వకు.
- `777` almost ఎప్పుడూ తప్పు (security hole) – నిజంగా అవసరమా అని ఆలోచించు.

Interview దృష్టి

Q: SSH private key కి `600` ఎందుకు తప్పనిసరి? A: `600` = owner కి మాత్రమే read/write, others కి ఏమీ లేదు. Key ని others చదవగలిగితే, ఆ server కి login చేయగలరు – huge security risk. అందుకే `~/.ssh/id_rsa` permissions loose గా (ఉదా 644) ఉంటే, SSH client "UNPROTECTED PRIVATE KEY FILE" అని error ఇచ్చి connect అవ్వదు.

Q: Directory కి `x` (execute) permission అంటే ఏమిటి? A: File లో `x` = run. కానీ directory లో `x` = దానిలోకి `cd` చేయడం / దాని ద్వారా paths access చేయడం. `r` లేకుండా `x` ఉంటే – పేర్లు తెలిస్తే access చేయగలవు కానీ `ls` చేయలేవు. Web folders కి `755` (x ఉంది) అందుకే – traverse చేయాలి కానీ list అవసరం లేదు.

Q: `sudo` మరియు `su` తేడా? A: `su` = మరో user గా పూర్తిగా switch (`su root` → root shell, ఆ user password కావాలి). `sudo` = ఒక్క command ని elevated గా run (నీ *own* password, `su` కాదు). `sudo` better – granular, logged, root password పంచుకోనవసరం లేదు. Production లో direct root login usually disable చేస్తారు.

5. Processes – ps, top, kill, jobs, systemctl

వివరణ

నీ Node app run చేస్తే, అది ఒక **process** అవుతుంది. Process = **currently running program** (memory + CPU వాడుతూ). ప్రతి process కి ఒక unique **PID (Process ID)** ఉంటుంది. SSE గా నువ్వు రోజూ చేసేవి: "ఏ processes run అవుతున్నాయి?", "port 3000 ఎవరు వాడుతున్నారు?", "hang అయిన process ని kill చెయ్", "server restart తర్వాత కూడా app run అవ్వాలి".

Process basics:

- **PID** = process ID (unique number).
- **PPID** = parent process ID (దీన్ని ఎవరు start చేశారు).
- **foreground** = terminal ని block చేసి run (నువ్వు `npm start` చేస్తే).
- **background** = terminal free గా ఉంచి run (`&` తో).
- **daemon** = background లో నిరంతరం run అయ్యే service (nginx, mongod, sshd – పేరు చివర `d`).

Real-life Scenario

> **Processes = ఒక hospital లో patients.**
>
> - `ps` / `top` = ward లో ఏ patients (processes) ఉన్నారో, ఎవరు ఎక్కువ resources (CPU/RAM = oxygen/bed) వాడుతున్నారో చూసే register.
> - **PID** = ప్రతి patient కి unique ID band.
> - `kill` = ఒక patient ని discharge చేయడం. `kill -15` = "దయచేసి బయటకి వెళ్ళండి" (polite, cleanup కి time). `kill -9` = security guard బలవంతంగా బయటకి తోసేయడం (instant, cleanup లేదు).
> - `systemctl` = hospital administration – "ఈ patient (service) ఎప్పుడూ ward లో ఉండాలి; discharge అయితే వెంటనే re-admit చెయ్" (auto-restart).
>
> Server లో "CPU 100% ఎందుకు?" అంటే `top` తెరిచి ఏ process culprit అని చూడటం – hospital లో "ఎవరు అల్లరి చేస్తున్నారు?" అని ward చూసినట్టు.

`ps` – process snapshot

```
ps aux                  # అన్ని processes (a=all users, u=details, x=daemons కూడా)
# USER   PID  %CPU %MEM    VSZ   RSS TTY  STAT START   TIME COMMAND
# ubuntu 1234  2.0  5.1 998000 52000 ?    Sl   10:00   0:15 node server.js
#        ^PID  ^CPU ^RAM              ^status             ^ఏ command

ps aux | grep node      # node processes మాత్రమే filter (అత్యంత common)
# ubuntu 1234  2.0  5.1 ... node server.js

pgrep -a node           # node processes యొక్క PIDs (grep కంటే clean)
# 1234 node server.js

ps -ef | grep mongod    # మరో style (System V) – PPID కూడా చూపిస్తుంది
```

`top` / `htop` – live process monitor

```
top                     # live updating — CPU, RAM, processes (Linux Task Manager)
#   keys: P=CPU sort, M=memory sort, k=kill (PID అడుగుతుంది), q=quit
#   పైన: load average, total tasks, %CPU, memory usage

htop                    # top యొక్క prettier version (colors, mouse, tree view)
                        # server లో లేకపోతే: sudo apt install htop
```

`top` header ముఖ్య metrics:

Metric	అర్థం
load average	1/5/15 min avg. cores కంటే ఎక్కువ = overloaded (4 cores లో 4.0 = full)
%CPU / %MEM	ఏ process ఎంత CPU/RAM వాడుతోంది
RES (RSS)	actual RAM వాడకం (ఇదే ముఖ్యం)
STAT	R=running, S=sleeping, Z=zombie, D=uninterruptible

`kill` – process ని ఆపడం + signals

`kill` నిజానికి process కి **signal** పంపుతుంది (చంపడం మాత్రమే కాదు):

Signal	Number	అర్థం	ఎప్పుడు
`SIGTERM`	15	polite "please stop" (cleanup కి time ఇస్తుంది)	**default**, ఎప్పుడూ మొదట ఇదే
`SIGKILL`	9	force kill (cleanup లేదు, instant)	SIGTERM పనిచేయకపోతే మాత్రమే
`SIGINT`	2	interrupt (`Ctrl+C` పంపేది)	foreground process ఆపడానికి
`SIGHUP`	1	hangup (చాలా daemons దీనితో **config reload** చేస్తాయి)	nginx reload

```
kill 1234               # PID 1234 కి SIGTERM (15) — graceful stop
kill -9 1234            # SIGKILL — force (hang అయిన process కి last resort)
kill -15 1234           # explicit SIGTERM
kill -HUP 1234          # config reload (kill కాదు!)
kill -l                 # అన్ని signals list

pkill node              # పేరుతో kill (అన్ని node processes)
pkill -f "server.js"    # -f = full command line match
killall node            # అదే — పేరుతో అన్నీ

# ⭐ "port 3000 already in use" — MERN classic problem:
lsof -i :3000           # port 3000 ఎవరు వాడుతున్నారో
# COMMAND  PID   USER   ...
# node    1234  ubuntu ...
kill -9 1234            # ఆ process ని kill → port free
```

```
# లేదా ఒకే line లో:
kill -9 $(lsof -t -i:3000)
```

> **Graceful shutdown ముఖ్యం:** ఎప్పుడూ మొదట `kill` (SIGTERM) try చెయ్ – Node app `process.on('SIGTERM')` catch చేసి, open DB connections close చేసి, in-flight requests పూర్తి చేసి, cleanly exit అవుతుంది. `kill -9` వెంటనే చంపేస్తుంది – data corruption, half-written files రిస్క్. `-9` ని *only* SIGTERM fail అయినప్పుడు.

Background jobs: `&`, `jobs`, `fg`, `bg`, `nohup`

```
# foreground లో run → terminal block అవుతుంది, Ctrl+C ఆపేస్తుంది
npm start

# background లో run → terminal free
npm start &              # & = background లో run
# [1] 1234               → job number [1], PID 1234

jobs                     # ఈ shell background jobs
# [1]+  Running    npm start &

fg %1                    # job 1 ని foreground కి తెచ్చు
bg %1                    # stopped job ని background లో continue

# Ctrl+Z = foreground job ని pause (suspend) → తర్వాత bg/fg తో resume

# ⚠️ ప్రధాన సమస్య: terminal/SSH close అయితే '&' background job కూడా చస్తుంది (SIGHUP).
# పరిష్కారం – nohup (no hangup):
nohup node server.js &                # terminal close అయినా బతికుంటుంది
# output → nohup.out file లో వెళ్తుంది
nohup node server.js > app.log 2>&1 &   # output ని app.log కి redirect (better)
```

> **నిజ ప్రపంచంలో:** MERN app ని `nohup ... &` తో run చేయరు (fragile – crash అయితే restart అవ్వదు). బదులుగా **process manager** వాడతారు: `pm2` (Node కి popular) లేదా `systemd` service. pm2: `pm2 start server.js`, `pm2 list`, `pm2 logs`, `pm2 restart`. Auto-restart on crash + boot.

`systemctl` – services (systemd)

Production లో apps/services ని **systemd** manage చేస్తుంది – auto-start on boot, auto-restart on crash, log management.

```
sudo systemctl status nginx      # service ఎలా ఉంది? (running/failed)
# ● nginx.service - A high performance web server
#    Active: active (running) since ...

sudo systemctl start nginx       # service start
sudo systemctl stop nginx        # stop
sudo systemctl restart nginx     # restart (stop + start)
sudo systemctl reload nginx      # config reload (downtime లేకుండా – restart కంటే
better)
```

```
sudo systemctl enable nginx      # boot అయినప్పుడు auto-start
sudo systemctl disable nginx     # auto-start ఆపు

systemctl list-units --type=service --state=running   # running services అన్నీ

# service logs (journald):
journalctl -u nginx              # nginx logs
journalctl -u myapp -f           # -f = live follow (tail -f లాంటిది)
journalctl -u myapp --since "10 min ago"
```

restart vs reload:

Command	ఏం చేస్తుంది	downtime
`restart`	process ని పూర్తిగా stop + start	క్షణం down
`reload`	config re-read, process అలాగే	zero downtime (nginx కి prefer)

Key Points

- **Process = running program**, ప్రతిదానికీ **PID**. `ps aux | grep <name>` తో వెతుకు.
- `top` / `htop` = live CPU/RAM monitor; load average > cores = overloaded.
- `kill <PID>` = SIGTERM (graceful, default); `kill -9` = SIGKILL (force, last resort only).
- **"port in use"** → `lsof -i :3000` → `kill` ఆ PID. MERN లో రోజూ.
- `&` = background (కానీ SSH close అయితే చస్తుంది); `nohup` / `pm2` / `systemd` = persistent.
- **systemd** production లో services manage చేస్తుంది: `systemctl start/stop/restart/enable`, `journalctl -f` logs.

Interview దృష్టి

Q: `kill -9` ఎందుకు last resort? A: `-9` (SIGKILL) ని process catch చేయలేదు – OS వెంటనే terminate చేస్తుంది. అంటే cleanup handler run అవ్వదు: open files half-written, DB transactions incomplete, locks release అవ్వవు, temp files మిగిలిపోతాయి. ఎప్పుడూ మొదట SIGTERM (graceful) – app cleanup చేసుకోనివ్వు. SIGTERM కి respond అవ్వకపోతేనే -9.

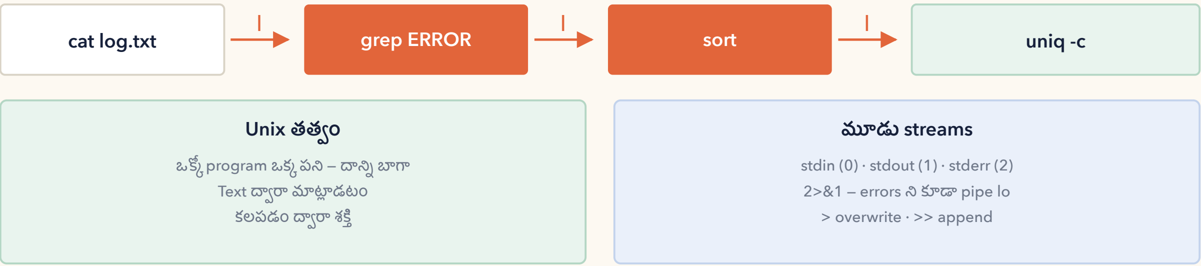
Q: Zombie process అంటే? A: Child process exit అయ్యింది కానీ parent దాని exit status ని ఇంకా "reap" (wait) చేయలేదు – process table లో `Z` state గా మిగిలిపోతుంది. Resources వాడదు కానీ PID slot ఆక్రమిస్తుంది. చాలా zombies = parent లో bug (wait() call లేదు). Parent ని kill/restart చేస్తే init/systemd వాటిని adopt చేసి reap చేస్తుంది.

Q: SSH close అయినా Node app run అవ్వాలంటే? A: `nohup node server.js &` (basic), కానీ production లో `pm2` లేదా `systemd` service – ఎందుకంటే అవి crash అయితే auto-restart, boot అయితే auto-start, log rotation, monitoring కూడా ఇస్తాయి. `nohup` కేవలం hangup signal ignore చేస్తుంది, crash recovery ఇవ్వదు.

6. Text Processing Power Tools — pipes, grep, sed, awk, find

PIPES · చిన్న సాధనాలని కలపడం

PIPES — ఒక program యొక్క output, తర్వాతి దానికి input



తరచుగా అవసరమయ్యే కలయిక

`grep -r "TODO" . | wc -l` — ఎన్ని TODOs ఉన్నాయి `ps aux | grep node | awk '{print $2}' | xargs kill` — అన్ని node processes ని ఆపడం

వివరణ

ఇది Linux యొక్క **అసలు superpower**. Unix philosophy: "**ఒక్కో tool ఒక్క పని చక్కగా చెయ్; వాటిని pipe తో కలిపి పెద్ద పనులు చెయ్**". ప్రతి tool text లో input తీసుకుని, text output ఇస్తుంది. వీటిని కలిపితే — logs analyze, data extract, bulk edit — అన్నీ ఒక్క line లో. SSE interview లో "ఈ log file నుండి top 10 IP addresses తీయి" లాంటివి ఇక్కడి skills తో solve చేస్తావు.

Real-life Scenario

> **Pipes = ఒక factory assembly line.**
>
> ఒక్కో machine (command) ఒక్క పని చేస్తుంది, తర్వాతి machine కి pass చేస్తుంది. Raw material (`cat log.txt`) → machine 1 (`grep ERROR` — errors మాత్రమే వడకట్టు) → machine 2 (`cut` — ఒక column తియ్యి) → machine 3 (`sort | uniq -c` — లెక్కపెట్టు) → finished product (top errors report).
>
> ప్రతి machine చిన్నదే, కానీ line గా కలిపితే powerful. ఒక్క machine అన్నీ చేయాలనుకుంటే complex; చిన్న tools + pipe = simple & flexible. ఇదే Unix philosophy. MERN లో production log నుండి "ఏ endpoint ఎక్కువ 500 errors ఇస్తోంది?" ఇలాంటి line తోనే కనిపెడతావు.

Pipes `|` — commands ని కలపడం

```
# ఒక command output → తర్వాతి command input
ls -l | grep ".js"              # .js files మాత్రమే
cat access.log | grep 404       # 404 lines
ps aux | grep node | wc -l      # node processes ఎన్ని (count)
history | grep git              # ముందు వాడిన git commands
```

pipe (`|`) = ఎడమ command యొక్క **stdout** ని కుడి command యొక్క **stdin** కి connect చేస్తుంది.

Redirection – `>`, `>>`, `<`, `2>`

ప్రతి process కి 3 **streams**: **stdin (0)** input, **stdout (1)** normal output, **stderr (2)** error output.

```
echo "hello" > file.txt         # > = stdout ని file లోకి (overwrite – ఉన్నది తుడుస్తుంది!)
echo "world" >> file.txt        # >> = append (చివర్లో కలుపు, తుడవదు)
node server.js > out.log        # app output ని file కి
node server.js 2> err.log       # 2> = stderr మాత్రమే file కి
node server.js > out.log 2>&1   # stdout + stderr రెండూ out.log కి (2>&1 = stderr ని
stdout కి merge)
node server.js &> all.log       # shortcut for రెండూ (bash)
node server.js < input.txt      # < = file ని stdin గా feed
command > /dev/null 2>&1        # అన్ని output ని పారేయి (silent run)
sort < names.txt                # file నుండి read (< usually optional)
```

Redirection cheat table:

Symbol	అర్థం
`>`	stdout → file (**overwrite**)
`>>`	stdout → file (**append**)
`2>`	stderr → file
`2>&1`	stderr ని stdout ఉన్న చోటికి పంపు
`&>`	stdout + stderr రెండూ (bash)
`<`	file → stdin
`/dev/null`	"black hole" – పంపిన data delete (output వద్దు అంటే)

> `2>&1` **order ముఖ్యం:** `> out.log 2>&1` పనిచేస్తుంది (stdout ని file కి పంపి, తర్వాత stderr ని అక్కడికే). కానీ `2>&1 > out.log` పనిచేయదు (stderr ముందే terminal కి bind అయ్యింది). Order left-to-right.

`grep` – text లో pattern వెతకడం (అత్యంత వాడే tool)

```
grep "ERROR" app.log            # ERROR ఉన్న lines
grep -i "error" app.log         # -i = case-insensitive (Error, ERROR, error అన్నీ)
grep -n "TODO" server.js        # -n = line numbers తో
grep -r "apiKey" src/           # -r = recursive (folder మొత్తం search) – code లో
వెతకడానికి classic
grep -v "DEBUG" app.log         # -v = invert (DEBUG *లేని* lines)
grep -c "404" access.log        # -c = count (ఎన్ని lines match అయ్యాయి)
grep -l "mongoose" src/*.js     # -l = matching filenames మాత్రమే
grep -A 3 "Exception" app.log   # -A 3 = match తర్వాత 3 lines కూడా (After)
grep -B 2 "Exception" app.log   # -B 2 = ముందు 2 lines (Before)
grep -C 2 "Exception" app.log   # -C 2 = ముందు+తర్వాత 2 lines (Context)
grep -w "id" file               # -w = whole word (id, "id" కాదు "video")
grep -E "error|warn|fatal" log  # -E = extended regex (egrep) – OR pattern
```

```
# combos:
ps aux | grep -i node           # running node processes
grep -rn "console.log" src/     # code లో మిగిలిపోయిన console.log లు (line numbers సహా)
tail -f app.log | grep --line-buffered ERROR   # live errors
```

`grep` vs `egrep`: `egrep` = `grep -E` (extended regex — `|`, `+`, `?`, `()` కి backslash అవసరం లేదు). ఆధునికంగా `grep -E` prefer చేస్తారు (`egrep` deprecated కానీ పనిచేస్తుంది).

`find` — files ని వెతకడం (grep = content; find = files)

```
find . -name "*.js"                    # ఇక్కడి నుండి అన్ని .js files
find . -iname "readme*"                # -iname = case-insensitive
find /var/log -name "*.log" -type f    # -type f = files మాత్రమే (d = dirs)
find . -type d -name "node_modules"    # node_modules folders వెతుకు
find . -mtime -1                       # 1 రోజులో modified అయినవి (-mtime -N)
find . -mtime +30                      # 30 రోజుల కంటే పాతవి
find . -size +100M                     # 100MB కంటే పెద్ద files (disk full debug!)
find . -empty                          # ఖాళీ files/folders

# ⭐ find + action (చాలా powerful):
find . -name "*.tmp" -delete           # అన్ని .tmp files delete
find . -name "*.log" -exec rm {} \;    # -exec: ప్రతి result మీద command ({} = filename)
find . -name "*.js" -exec grep -l "TODO" {} \;   # .js files లో TODO ఉన్నవి
find /tmp -mtime +7 -delete            # 7 రోజుల కంటే పాత temp files clean (cron
classic)
```

`sed` — stream editor (find & replace, delete lines)

```
sed 's/old/new/' file.txt          # ప్రతి line లో మొదటి "old" → "new" (print only)
sed 's/old/new/g' file.txt         # g = global (line లో అన్నీ)
sed -i 's/localhost/127.0.0.1/g' config.txt    # -i = in-place (file ని నిజంగా మార్చు!)
sed -i.bak 's/8080/3000/g' .env    # -i.bak = మార్చే ముందు .env.bak backup తీయి
sed -n '10,20p' file.txt           # 10-20 lines మాత్రమే print (-n = quiet, p = print)
sed '5d' file.txt                  # 5వ line delete
sed '/DEBUG/d' app.log             # DEBUG ఉన్న lines అన్నీ delete
echo "hello world" | sed 's/world/Linux/'   # hello Linux
```

⚠️ **`sed -i` జాగ్రత్త:** file ని నేరుగా మారుస్తుంది (undo లేదు). ముఖ్యమైన file అయితే `-i.bak` వాడి backup తీయి, లేదా ముందు `-i` లేకుండా run చేసి output verify చేయి. macOS లో `sed -i ''` (ఖాళీ arg) కావాలి — GNU vs BSD తేడా.

`awk` — column-based processing (data extract, reports)

`awk` = ప్రతి line ని columns గా చూస్తుంది (`$1` =1st, `$2` =2nd... `$0` =whole line). Space/tab default separator.

```
awk '{print $1}' access.log        # ప్రతి line మొదటి column (log లో usually IP)
awk '{print $1, $4}' file          # 1st + 4th columns
awk -F: '{print $1}' /etc/passwd   # -F: = colon separator → usernames
```

```
ls -l | awk '{print $5, $9}'       # file size + name
awk '{sum += $5} END {print sum}' data.txt   # 5వ column sum (total size లెక్క)
awk '$3 > 100 {print $1}' data     # 3వ column 100 కంటే ఎక్కువ ఉన్న rows లో 1వ column
awk 'NR==1 {next} {print}' csv     # header (line 1) skip చేయు (NR = line number)
awk -F, '{print $2}' data.csv      # CSV 2వ column
df -h | awk '$5+0 > 80 {print $6}' # disk 80% పైన ఉన్న mount points!
```

`wc`, `sort`, `uniq`, `cut`, `xargs` — chorus of helpers

```
# --- wc: count ---
wc -l file.txt          # lines count (అత్యంత common — "ఎన్ని lines?")
wc -w file.txt          # words
wc -c file.txt          # bytes
ls | wc -l              # ఈ folder లో ఎన్ని items

# --- sort ---
sort names.txt          # alphabetical
sort -r names.txt       # reverse
sort -n numbers.txt     # numeric (10 < 9 కాకుండా సరిగ్గా)
sort -u names.txt       # sort + unique (duplicates తీసేయి)
du -sh * | sort -h      # -h = human sizes sort (పెద్ద folders కనిపెట్టడానికి)

# --- uniq (⚠️ ముందు sort అవసరం — adjacent duplicates మాత్రమే చూస్తుంది) ---
sort file | uniq        # duplicates తీసేయి
sort file | uniq -c     # -c = ప్రతిదాని count తో
sort file | uniq -d     # duplicates మాత్రమే
sort file | uniq -c | sort -rn   # ⭐ frequency count, ఎక్కువ నుండి తక్కువ

# --- cut: column/field extract ---
cut -d, -f1 data.csv    # -d, = comma delimiter, -f1 = field 1
cut -d: -f1 /etc/passwd # usernames
cut -c1-10 file         # ప్రతి line మొదటి 10 characters
echo "a:b:c" | cut -d: -f2   # b

# --- xargs: ఒక command output ని తర్వాతి command *arguments* గా మార్చు ---
find . -name "*.log" | xargs rm          # అన్ని .log files delete
cat urls.txt | xargs -n1 curl -O         # ప్రతి URL download (-n1 = ఒక్కొక్కటి)
find . -name "*.js" | xargs grep "TODO"  # .js files అన్నిటిలో TODO
git branch --merged | grep -v main | xargs git branch -d  # merged branches cleanup
```

> `|` **vs** `xargs` **తేడా:** `echo "file.txt" | rm` పనిచేయదు — `rm` stdin చదవదు, arguments కావాలి. `echo "file.txt" | xargs rm` పనిచేస్తుంది — `xargs` stdin ని arguments గా మార్చి `rm file.txt` run చేస్తుంది. Filenames లో spaces ఉంటే `find ... -print0 | xargs -0` వాడు (null-separated, safe).

⭐ Real SSE combos — logs analysis (interview favorites)

```
# nginx access log నుండి top 10 IP addresses (traffic ఎక్కడి నుండి)
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
# 4521 203.0.113.5
# 3210 198.51.100.2 ...

# HTTP status codes distribution (ఎన్ని 200, 404, 500)
```

```
awk '{print $9}' access.log | sort | uniq -c | sort -rn
# 45201 200
#   320 404
#    12 500

# ఒక endpoint కి 500 errors ఎన్నిసార్లు
grep " 500 " access.log | grep "/api/checkout" | wc -l

# app.log లో గత గంటలో ERROR లు, unique messages తో
grep ERROR app.log | awk -F']' '{print $2}' | sort | uniq -c | sort -rn
```

Tool ఎంపిక – ఏ పనికి ఏది

పని	Tool
Text లో pattern వెతకడం	`grep`
Files (పేరు/size/date) వెతకడం	`find`
Find & replace, line delete	`sed`
Columns extract, math, reports	`awk` (లేదా simple అయితే `cut`)
Lines/words count	`wc`
Sort, duplicates, frequency	`sort` + `uniq -c`
Output → next command args	`xargs`

Key Points

- **Pipe `|`** = ఒక command output → తర్వాతిదాని input. Unix superpower – చిన్న tools కలిపి పెద్ద పని.
- **Rediection:** `>` overwrite, `>>` append, `2>` errors, `2>&1` merge, `/dev/null` = discard.
- **`grep`** = content search (`-r` recursive, `-i` case, `-v` invert, `-n` line#, `-E` regex). **`find`** = file search.
- **`sed`** = find/replace & delete (`-i` in-place – జాగ్రత్త). **`awk`** = column extraction & math (`$1 $2 ...`).
- **`sort | uniq -c | sort -rn`** = frequency count combo – logs analysis backbone.
- **`xargs`** = stdin ని arguments గా మార్చి తర్వాతి command కి pass.

Interview దృష్టి

Q: `grep` and `find` తేడా? A: `find` = **files** ని పేరు/size/date/type ప్రకారం వెతుకుతుంది (filesystem). `grep` = files లోపల **content** (text pattern) వెతుకుతుంది. చాలాసార్లు కలిపి వాడతారు: `find . -name "*.js" | xargs grep "TODO"` – .js files కనిపెట్టి, వాటిలో TODO వెతకడం.

Q: `sed` ఎలా పనిచేస్తుంది, `-i` risk ఏమిటి? A: `sed` = stream editor – input ని line-by-line చదివి, transformation apply చేసి, output ఇస్తుంది. Default గా original ముట్టదు (stdout కి print). `-i` = in-

place – file ని నిజంగా overwrite చేస్తుంది, **undo లేదు**. Production files కి `-i.bak` తో backup, లేదా ముందు `-i` లేకుండా run చేసి verify.

Q: log file నుండి top 10 error-producing IPs ఎలా తీస్తావ్? A: `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10`. Logic: `awk` IP column తీస్తుంది → `sort` వాటిని group అవ్వడానికి పక్కపక్కన పెడుతుంది → `uniq -c` count → `sort -rn` ఎక్కువ నుండి → `head -10` top 10. ఇది Unix philosophy కి perfect example.

---

7. Networking & Remote – curl, ssh, scp, rsync

వివరణ

MERN developer గా నీ app remote server మీద run అవుతుంది. దాన్ని reach అవ్వడానికి (**ssh**), files transfer చేయడానికి (**scp/rsync**), API test చేయడానికి (**curl**), "port open ఉందా, service reachable నా?" చూడటానికి (**ping, ss**) – ఈ networking commands తప్పనిసరి. Interview లో "server కి ఎలా connect అవుతావ్?", "API endpoint ఎలా test చేస్తావ్ terminal నుండి?" – ఇవి ఇక్కడ.

Real-life Scenario

> **Remote server access = వేరే ఊరిలో ఉన్న ఇంటిని manage చేయడం.**
>
> - **ssh** = ఆ ఇంటికి secure గా వెళ్ళే teleport door (encrypted). Key (SSH key) ఉంటేనే తెరుచుకుంటుంది.
> - **scp/rsync** = ఆ ఇంటికి సామాన్లు (files) పంపే courier. `rsync` smart courier – ఏది మారిందో అదే పంపుతుంది.
> - **curl** = door దగ్గర నిలబడి "ఇంట్లో ఎవరైనా ఉన్నారా? ఏం response ఇస్తారు?" అని knock చేయడం (HTTP request).
> - **ping** = "ఆ ఇల్లు అసలు existent గా ఉందా, reachable నా?" అని echo పంపడం.
> - **ss/netstat** = ఇంటి లోపల ఏ ఏ తలుపులు (ports) తెరిచి ఉన్నాయో చూడటం.
>
> Production deploy = ssh తో server కి వెళ్ళి, rsync తో code పంపి, curl తో "app బతికుందా?" అని health-check చేయడం.

`curl` & `wget` – HTTP requests, downloads

`curl` = command line HTTP client. API testing, health checks, webhooks – SSE రోజూ వాడతాడు.

```
curl https://api.example.com/users          # GET request (response terminal కి)
curl -i https://api.example.com/health       # -i = response headers కూడా చూపు
curl -I https://example.com                   # -I = headers మాత్రమే (HEAD request)
curl -s https://api.example.com/data          # -s = silent (progress bar వద్దు,
scripts కి)
curl -L https://bit.ly/xyz                    # -L = redirects follow (301/302)
curl -o page.html https://example.com         # -o = output ని file కి
curl -O https://example.com/file.zip          # -O = remote filename తోనే save
```

```
# POST request (API test — MERN backend testing కి classic):
curl -X POST https://api.example.com/login \
  -H "Content-Type: application/json" \
  -d '{"email":"a@b.com","password":"123"}'
#  -X = method, -H = header, -d = body data

# Authorization header తో (JWT token):
curl -H "Authorization: Bearer eyJhb...token" https://api.example.com/me

# health check pattern (deploy తర్వాత "app up అయ్యిందా?"):
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/health
# 200      (body పారేసి, HTTP status మాత్రమే — scripts లో perfect)

# --- wget: download-focused ---
wget https://example.com/file.zip              # download (curl -O లాంటిది)
wget -c https://example.com/big.iso            # -c = continue (interrupted download resume)
wget -r https://example.com/                   # -r = recursive (whole site mirror)
```

curl vs wget:

అంశం	`curl`	`wget`
ప్రధాన పని	API interaction (GET/POST/headers)	file download
output	default terminal కి	default file కి
recursive download	❌	✅ (`-r`)
resume download	`-C -`	✅ (`-c`)
SSE use	API testing, health checks, webhooks	files/assets download

`ping` — host reachable నా?

```
ping google.com              # continuous echo (Ctrl+C to stop)
# 64 bytes from ... time=12.3 ms    ← reachable, latency 12ms
ping -c 4 google.com         # -c 4 = 4 packets మాత్రమే
# 4 packets transmitted, 4 received, 0% packet loss

ping 8.8.8.8                 # IP తో (DNS problem ఉందా అని test — domain fail, IP pass = DNS issue)
```

> **Debug logic:** `ping 8.8.8.8` పనిచేసి `ping google.com` fail అయితే → network fine కానీ **DNS** problem. రెండూ fail అయితే → network/connectivity issue. (కొన్ని servers ping/ICMP block చేస్తాయి — ping fail అయినంత మాత్రాన server down అని కాదు.)

`ss` / `netstat` – ఏ ports open, ఏ connections

```
ss -tulpn              # అత్యంత common: t=TCP, u=UDP, l=listening, p=process,
n=numbers
# Netid State  Local Address:Port  Process
# tcp   LISTEN 0.0.0.0:3000        node (pid=1234)    ← Node app port 3000 లో వింటోంది
# tcp   LISTEN 0.0.0.0:80          nginx (pid=567)
# tcp   LISTEN 127.0.0.1:27017     mongod             ← Mongo localhost మాత్రమే

ss -tulpn | grep 3000   # port 3000 ఎవరు వాడుతున్నారు
netstat -tulpn          # పాత tool, అదే output (కొన్ని servers లో ss లేకపోతే)

# "నా app ఏ port లో వింటోంది?" లేదా "port already in use ఎవరిది?" – వీటికి ఇదే
```

> **`ss` vs `netstat`:** `ss` (socket statistics) = ఆధునికం, వేగం. `netstat` = పాతది, deprecating కానీ ఇంకా ఎక్కడో ఉంటుంది. రెండూ ఒకటే info. `0.0.0.0:3000` = అన్ని interfaces మీద (బయటి నుండి reachable); `127.0.0.1:3000` = localhost మాత్రమే (బయటి నుండి కాదు).

`ssh` – remote server కి secure login (⭐ SSE కి core)

```
ssh ubuntu@203.0.113.5              # user@server-ip తో login
ssh ubuntu@myserver.com             # domain తో
ssh -i ~/.ssh/mykey.pem ubuntu@ec2-x.amazonaws.com   # -i = specific key (AWS EC2
classic)
ssh -p 2222 ubuntu@server           # -p = non-default port (default 22)

# SSH లోకి వెళ్ళాక server లో ఉన్నట్టే – commands run చెయ్, exit తో బయటకి
exit                                 # SSH session ముగించు

# ఒక్క command run చేసి బయటకి రావడం (SSH లోకి వెళ్ళకుండా):
ssh ubuntu@server "df -h"            # server disk చూసి వచ్చేయి
ssh ubuntu@server "pm2 restart app"  # remote గా app restart

# SSH config (~/.ssh/config) – పొడవు commands ని alias చేయడం:
# ఈ file లో:
#   Host prod
#     HostName 203.0.113.5
#     User ubuntu
#     IdentityFile ~/.ssh/mykey.pem
#     Port 22
# ఇప్పుడు కేవలం:
ssh prod                             # పైనది అంతా automatic!
```

SSH keys – password కంటే secure (ఎలా setup):

```
ssh-keygen -t ed25519 -C "your@email.com"   # key pair generate (private + public)
# → ~/.ssh/id_ed25519 (private – ⚠️ ఎవరికీ ఇవ్వకు), ~/.ssh/id_ed25519.pub (public)

ssh-copy-id ubuntu@server           # public key ని server కి copy (ఇక password అవసరం
```

```
లేదు)
# లేదా manual: public key content ని server లో ~/.ssh/authorized_keys కి కలుపు
```

> **SSH key model:** private key నీ దగ్గర (గోప్యం, `600` permission), public key server లో. Server encrypted challenge పంపుతుంది, నీ private key తో మాత్రమే decrypt/sign అవుతుంది → password లేకుండా secure login. GitHub కూడా ఇలాగే – SSH key add చేస్తే `git push` కి password అడగదు.

`scp` & `rsync` – files transfer

```
# --- scp: secure copy (SSH మీద file transfer) ---
scp app.zip ubuntu@server:/home/ubuntu/       # local → remote
scp ubuntu@server:/var/log/app.log ./         # remote → local (logs తెచ్చుకోవడం)
scp -r dist/ ubuntu@server:/var/www/app/       # -r = folder (recursive)
scp -i key.pem file ubuntu@server:~/           # -i = key తో

# --- rsync: smart sync (మారిన భాగం మాత్రమే – deploy కి best) ---
rsync -avz dist/ ubuntu@server:/var/www/app/
#      -a = archive (permissions/timestamps preserve), -v = verbose, -z = compress
rsync -avz --delete dist/ ubuntu@server:/var/www/app/
#      --delete = source లో లేని files ని destination లో కూడా తీసేయి (exact mirror)
rsync -avz --exclude 'node_modules' --exclude '.git' ./ ubuntu@server:~/app/
#      --exclude = ఇవి skip (node_modules server లో npm install చేస్తాం)
rsync -avzP big.zip ubuntu@server:~/           # -P = progress + resume
```

scp vs rsync:

అంశం	`scp`	`rsync`
transfer	ప్రతిసారి అన్నీ మళ్ళీ	**మారిన భాగం మాత్రమే** (delta)
speed (repeated)	slow	చాలా fast
resume	❌	✅ (`-P`)
exclude/mirror	❌	✅ (`--exclude` , `--delete`)
ఎప్పుడు	ఒక్కసారి చిన్న file	deploy, repeated sync, పెద్ద folders

> **MERN deploy pattern:** `rsync -avz --exclude 'node_modules' --exclude '.git' ./ prod:~/app/` తో code పంపి, తర్వాత `ssh prod "cd ~/app && npm ci && pm2 restart app"` . rsync వల్ల ప్రతిసారి మారిన files మాత్రమే వెళ్తాయి – deploy fast.

Key Points

- **`curl`** = API testing/health checks (`-X POST -H -d` , `-w "%{http_code}"`); **`wget`** = downloads (`-c` resume).
- **`ping`** = reachability + DNS debug (`ping 8.8.8.8` pass, domain fail = DNS issue).

- `ss -tulpn` = ఏ ports open, ఏ process – "port in use" debug కి.
- `ssh user@host` = remote login; **SSH keys** (private local `600`, public on server) = password-less secure access. `~/.ssh/config` తో aliases.
- `scp` = one-off copy; `rsync -avz` = smart delta sync (deploy కి best, `--exclude`, `--delete`).

Interview దృష్టి

Q: SSH ఎలా secure గా authenticate చేస్తుంది? A: Asymmetric cryptography (key pair). Private key client దగ్గర, public key server లో (`authorized_keys`). Login సమయంలో server random challenge పంపుతుంది, client దాన్ని private key తో sign చేస్తుంది, server public key తో verify చేస్తుంది. Private key ఎప్పుడూ network మీద వెళ్ళదు – అందుకే password కంటే secure. మొత్తం session encrypted.

Q: `scp` కంటే `rsync` ఎందుకు better deploy కి? A: `rsync` delta-transfer చేస్తుంది – source, destination compare చేసి **మారిన/కొత్త files మాత్రమే** పంపుతుంది (checksums/timestamps తో). పెద్ద codebase లో మళ్ళీ deploy చేస్తే scp అంతా మళ్ళీ పంపుతుంది (slow); rsync కొన్ని KB మాత్రమే. Plus `--exclude` (node_modules skip), `--delete` (exact mirror), resume – deploy కి ideal.

Q: `curl` తో API health check ఎలా, script లో? A: `curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/health` – body ని `/dev/null` కి పారేసి, HTTP status code మాత్రమే print. Deploy script లో దీన్ని `if [ "$code" = "200" ]` తో check చేసి, fail అయితే rollback. Retry loop తో "app up అయ్యేదాకా wait" కూడా చేస్తారు.

---

8. Environment & Packages – env vars, PATH, .bashrc, apt/brew

వివరణ

Environment variables = shell/programs కి అందుబాటులో ఉండే key=value settings. MERN లో ఇవి రోజూ: `NODE_ENV=production`, `PORT=3000`, `DATABASE_URL=...`, `JWT_SECRET=...`. **PATH** = shell commands ని ఎక్కడ వెతకాలో చెప్పే special variable. **Package managers** (`apt`, `yum`, `brew`) = software install చేసే tools. ఇవి అర్థం చేసుకుంటే "`node: command not found` ఎందుకు వస్తోంది?", "config ని hardcode కాకుండా env నుండి ఎలా చదవాలి?" clear అవుతుంది.

Real-life Scenario

> **Environment variables = ఒక company యొక్క shared notice board.**
>
> Board మీద "OFFICE_LOCATION=Hyderabad", "MODE=production" లాంటివి రాసి ఉంటాయి. ఏ employee (program) అయినా board చూసి పని చేస్తారు – code లో hardcode చేయకుండా. Manager board మీద "MODE=maintenance" అని మార్చితే, అందరూ ఆ ప్రకారం behave చేస్తారు.
>
> **PATH = office లో tools ఎక్కడ ఉన్నాయో చెప్పే directory list.** "hammer కావాలంటే మొదట store-room-1 చూడు, లేకపోతే store-room-2..." అని ఒక priority list. `node` type చేస్తే shell PATH లోని folders ని ఆ order లో వెతికి, మొదట దొరికిన `node` ని run చేస్తుంది. అందుకే `nvm` node version మార్చినప్పుడు PATH మారుస్తుంది.

Environment variables – చూడటం, set చేయడం

```
env                      # అన్ని environment variables list
printenv                 # అదే
echo $HOME               # ఒక variable విలువ (PATH, USER, HOME...)
# /home/ubuntu
echo $PATH               # command search paths (colon separated)
# /usr/local/bin:/usr/bin:/bin:/home/ubuntu/.nvm/versions/node/v20/bin
echo $NODE_ENV           # (set అయితే) production
echo $USER $SHELL $PWD   # multiple

# --- set చేయడం ---
NAME="Yaswanth"          # current shell కి మాత్రమే (child processes కి కాదు)
echo $NAME               # Yaswanth

export NODE_ENV=production   # export = child processes కి కూడా అందుతుంది (⭐ ముఖ్యం)
export PORT=3000
node server.js           # ఈ node process కి NODE_ENV, PORT అందుతాయి

# ఒక్క command కి మాత్రమే (temporary, inline):
NODE_ENV=production PORT=8080 node server.js   # ఈ run కి మాత్రమే

unset NAME               # variable తీసేయడం
```

> **`NAME=x` vs `export NAME=x` తేడా (interview లో అడుగుతారు):** simple `NAME=x` = ప్రస్తుత shell లో మాత్రమే ఉంటుంది. నువ్వు `node` run చేస్తే ఆ child process కి కనిపించదు. `export NAME=x` = ఈ shell + అది spawn చేసే **అన్ని child processes** కి కూడా అందుతుంది. Node app లో `process.env.NAME` చదవాలంటే `export` (లేదా inline) తప్పనిసరి.

`.env` files (MERN లో standard)

Production లో secrets ని shell లో export చేయరు – `.env` file వాడతారు (`dotenv` package):

```
# .env file (⚠️ .gitignore లో పెట్టు – commit చేయకు!)
PORT=3000
DATABASE_URL=mongodb://localhost:27017/myapp
```

```
JWT_SECRET=super-secret-key
NODE_ENV=production
```

```
// server.js లో:
require('dotenv').config();          // .env ని process.env లోకి load చేస్తుంది
const port = process.env.PORT || 3000;
```

> **Security rule:** `.env` ఎప్పుడూ **git లో commit చేయకు** (secrets leak). బదులుగా `.env.example` (dummy values తో) commit చేసి, actual `.env` ని `.gitignore` లో పెట్టు. Server లో `.env` permissions `600`.

PATH – commands ఎలా దొరుకుతాయి

`PATH` = colon-separated directory list. నువ్వు `node` type చేస్తే, shell ఈ folders ని **ఎడమ నుండి కుడికి** వెతికి, మొదట దొరికిన `node` ని run చేస్తుంది.

```
echo $PATH
# /home/ubuntu/.nvm/.../bin:/usr/local/bin:/usr/bin:/bin

which node              # ఏ node run అవుతోంది (PATH లో మొదట దొరికినది)
# /home/ubuntu/.nvm/versions/node/v20.10.0/bin/node
which -a node           # -a = అన్ని matches (multiple installs ఉంటే)
type node               # which లాంటిది + alias/builtin కూడా చెప్తుంది
command -v node         # scripts లో "node ఉందా?" check చేయడానికి portable

# PATH కి కొత్త folder add (నీ scripts folder ని commands గా వాడటానికి):
export PATH="$HOME/bin:$PATH"       # ముందు add → priority ఎక్కువ
export PATH="$PATH:$HOME/bin"       # చివర add → priority తక్కువ
```

> **"command not found" debug:** `node: command not found` వస్తే → అది PATH లో ఏ folder లోనూ లేదు. `which node` ఖాళీ → install కాలేదు, లేదా install అయిన folder PATH లో లేదు. `nvm` వాడితే terminal restart చేయాలి (`.bashrc` reload). ఇది self-taught devs కి రోజూ వచ్చే confusion.

Shell startup files: `.bashrc`, `.zshrc`, `.profile`

కొత్త terminal తెరిచినప్పుడు shell ఈ files ని run చేస్తుంది – permanent settings, aliases, PATH ఇక్కడ పెడతారు.

File	ఎప్పుడు run అవుతుంది
`~/.bashrc`	ప్రతి కొత్త **interactive bash** shell (terminal తెరిచినప్పుడు)
`~/.bash_profile` / `~/.profile`	**login** shells (SSH login, తొలి login)
`~/.zshrc`	zsh కి (macOS default) – `.bashrc` కి equivalent

```
# ~/.bashrc లో పెట్టేవి (permanent):
export PATH="$HOME/bin:$PATH"
export NODE_ENV=development
alias ll='ls -la'                     # short commands (alias!)
alias gs='git status'
alias gp='git push'
alias ..='cd ..'
alias serve='python3 -m http.server'

# .bashrc మార్చాక — కొత్త terminal, లేదా reload:
source ~/.bashrc                      # ప్రస్తుత shell లో re-run (terminal restart అవసరం
లేదు)
. ~/.bashrc                           # source కి shortcut (.)
```

> **Aliases = నీ productivity booster.** రోజూ `git status` type చేయడం బదులు `gs`. SSE లు తమ `.bashrc` / `.zshrc` లో 20-30 aliases పెట్టుకుంటారు. కొత్త machine కి వెళ్ళినా ఈ file తీసుకెళ్తే setup ready (అందుకే చాలామంది "dotfiles" ని git repo లో ఉంచుతారు).

Package managers – software install

Manager	Distro/OS	commands
apt	Debian/Ubuntu	`apt update`, `apt install`, `apt remove`
yum / **dnf**	RHEL/CentOS/Amazon Linux	`yum install`, `dnf install`
brew	macOS (& Linux)	`brew install`, `brew upgrade`
apk	Alpine (Docker)	`apk add`

```
# --- apt (Ubuntu/Debian — servers లో ఎక్కువ) ---
sudo apt update                 # package list refresh (install ముందు ఎప్పుడూ ఇది!)
sudo apt upgrade                # installed packages ని update
sudo apt install nginx          # install
sudo apt install -y htop git curl   # -y = అన్ని prompts కి yes (scripts లో)
sudo apt remove nginx           # uninstall
sudo apt search nodejs          # వెతకడం
apt list --installed            # installed packages

# --- yum/dnf (RHEL/CentOS/Amazon Linux) ---
sudo yum install -y git
sudo yum update

# --- brew (macOS) ---
brew install node               # sudo అవసరం లేదు!
brew upgrade node
brew list                       # installed
brew search postgres
```

> **`apt update` vs `apt upgrade` గందరగోళం:** `apt update` = అందుబాటులో ఉన్న packages **list** ని refresh చేస్తుంది (ఏమీ install చేయదు). `apt upgrade` = నిజంగా packages ని కొత్త versions కి update చేస్తుంది. ఎప్పుడూ install ముందు `apt update` run చెయ్ – లేకపోతే "package not found" లేదా పాత version వస్తుంది.

`man` & `--help` – help ఎలా చూడాలి

```
man ls                    # ls యొక్క పూర్తి manual (q to quit, / to search)
man grep                  # ఏ command కైనా full docs
ls --help                 # quick help (man కంటే చిన్నది, వేగం)
tldr tar                  # (install అవసరం) – man కి simplified, examples-first version
apropos "copy files"      # keyword తో commands వెతకడం
```

> **SSE habit:** flag గుర్తు లేకపోతే `man <cmd>` లేదా `<cmd> --help`. Google కంటే వేగం, offline పనిచేస్తుంది. `man` లో `/pattern` తో search, `n` తో next.

Key Points

- **Env vars** = key=value settings programs చదువుతాయి. `echo $VAR` చూడటం, `export VAR=x` set చేయడం (child processes కి కూడా).
- `NAME=x` current shell కి మాత్రమే; **`export`** child processes కి కూడా – Node **`process.env`** కి తప్పనిసరి.
- **`.env` files** = MERN config/secrets; ⚠️ **git లో commit చేయకు**, `.gitignore` లో పెట్టు, `.env.example` commit.
- **PATH** = command search folders (ఎడమ నుండి). `which node` = ఏ node run అవుతోంది; "command not found" = PATH లో లేదు.
- **`.bashrc` / `.zshrc`** = permanent settings + **aliases** (productivity). మార్చాక `source ~/.bashrc`.
- Package managers: **apt** (Ubuntu), **yum** (RHEL), **brew** (mac). Install ముందు `apt update`.

Interview దృష్టి

Q: `export` ఎందుకు అవసరం, లేకపోతే ఏమవుతుంది? A: `export` లేకుండా variable ప్రస్తుత shell కి మాత్రమే visible – దాని child processes (నువ్వు run చేసే node, python) కి కనిపించదు. `export VAR=x` చేస్తే child processes కి inherit అవుతుంది. అందుకే `NODE_ENV` ని app చదవాలంటే `export NODE_ENV=production` (లేదా inline `NODE_ENV=production node app.js`) తప్పనిసరి.

Q: `node: command not found` – ఎలా debug చేస్తావ్? A: (1) `which node` ఖాళీ నా చూస్తా – ఖాళీ అయితే PATH లో లేదు. (2) node install అయిందా, ఏ folder లో? (3) ఆ folder PATH లో ఉందా (`echo $PATH`)? (4) nvm వాడితే `.bashrc` లో nvm init line ఉందా, terminal restart చేశానా? చాలాసార్లు fix = `source ~/.bashrc` లేదా PATH కి node folder add.

Q: Config ని code లో hardcode ఎందుకు చేయకూడదు? A: Environment ప్రకారం config మారుతుంది (dev DB vs prod DB), secrets code లో ఉంటే git ద్వారా leak అవుతాయి, ఒకే code ని అన్ని

environments లో deploy చేయలేం. **12-factor app** principle: config ని environment variables లో పెట్టు. అదే code, వేరే `.env` → dev/staging/prod. Secrets code base నుండి వేరు.

Part 2 – Shell Scripting

> ఇప్పటివరకు commands ఒక్కొక్కటి type చేశాం. కానీ ఒకే పని పదేపదే చేయాల్సి వస్తే? (ఉదా: "code pull చెయ్, npm install, build, restart" – ప్రతి deploy కి 5 commands). వాటిని ఒక **shell script** (`.sh` file) లో రాసి, ఒక్క command తో run చేయడం = **automation**. DevOps, CI/CD, deploy scripts, cron jobs – అన్నీ shell scripts. SSE గా నువ్వు scripts *రాయగలగాలి*, కనీసం *చదవగలగాలి* (ఎవరో రాసిన deploy script debug చేయడానికి). ఈ Part లో scripting ని scratch నుండి – variables, conditions, loops, functions, ఒక real deploy script వరకు.

---

9. Shell Scripting – automation యొక్క గుండె

SHELL SCRIPTING · SET -EUO PIPEFAIL

SHELL SCRIPT – safety header

set -euo pipefail

ప్రతి production script మొదటి line – ఇది లేకపోతే తప్పులు నిశ్శబ్దంగా దాటిపోతాయి

`-e`	ఏదైనా command fail అయితే వెంటనే ఆగడం
`-u`	నిర్వచించని variable వాడితే error
`-o pipefail`	pipe lo ఏ భాగం fail అయినా మొత్తం fail

ఇది లేకపోతే ఏమవుతుంది

`cd /nonexistent; rm -rf *` – cd fail అయినా rm నడుస్తుంది. మీ ప్రస్తుత directory lo. `echo "$UNDEFINED_VAR"` – ఖాళీ string, script పని చేసినట్టే కనిపిస్తుంది.ఈ ఒక్క line చాలా విపత్తులని ఆపుతుంది.

వివరణ

Shell script = ఒక file లో వరుసగా రాసిన shell commands + logic (conditions, loops, variables). దాన్ని run చేస్తే, commands ఒక్కొక్కటి execute అవుతాయి. మనిషి type చేయాల్సిన repetitive పనిని ఒకసారి రాసి పదేపదే run చేయవచ్చు – errors తగ్గుతాయి, వేగం పెరుగుతుంది.

Real-life Scenario

> **Shell script = ఒక వంట recipe కార్డ్.**
>
> రోజూ అదే వంట చేయడానికి ప్రతిసారి ఆలోచించడం బదులు – recipe card (script) రాసుకుంటావు: "1. ఉల్లి తరుగు, 2. నూనె వేడి చెయ్, 3. వేయించు...". ఇప్పుడు ఎవరైనా (లేదా నువ్వే మళ్ళీ) card చూసి అదే వంట exactly చేయగలరు. **Variables** = "ఎంత మిర్చి?" లాంటి adjustable ingredients. **if condition** = "కారం ఇష్టమైతే మిర్చి ఎక్కువ". **loop** = "5 రొట్టెలు చెయ్" (అదే steps 5 సార్లు).
>
> Deploy script = "code తెచ్చు → install → build → restart → health check" recipe. ఒకసారి రాస్తే, ప్రతి release కి ఒక్క command. మనిషి తప్పు చేసే chance తగ్గుతుంది.

మొదటి script – shebang & run

```
#!/bin/bash
# ↑ shebang: ఈ file ని ఏ interpreter తో run చేయాలో OS కి చెప్తుంది (/bin/bash)
# '#' తో మొదలయ్యేవి comments (shebang తప్ప)

echo "Hello, SSE!"
echo "ఇవాళ: $(date)"      # $(...) = command substitution – command output ని embed
```

```
# run చేయడానికి 2 మార్గాలు:
bash hello.sh              # (1) bash తో నేరుగా – execute permission అవసరం లేదు

chmod +x hello.sh          # (2) execute permission ఇచ్చి...
./hello.sh                 # ...నేరుగా run (shebang line ఏ interpreter అని చెప్తుంది)
# Hello, SSE!
# ఇవాళ: Thu Jul 17 14:30:00 UTC 2026
```

> **Shebang (`#!`) ఎందుకు?** File ని `./script.sh` గా run చేసినప్పుడు, OS మొదటి line చూసి "ఈ file ని `/bin/bash` తో run చెయ్" అని తెలుసుకుంటుంది. Python script అయితే `#!/usr/bin/env python3`. Shebang లేకపోతే OS ఏ interpreter వాడాలో తెలియదు. `env` వాడటం better (`#!/usr/bin/env bash`) – bash ఎక్కడున్నా PATH నుండి కనిపెడుతుంది.

Variables & Quoting (⭐ chief source of bugs)

```
#!/bin/bash
name="Yaswanth"           # ⚠️ = చుట్టూ SPACE ఉండకూడదు! (name = "x" తప్పు)
count=5
greeting="Hello, $name"   # variable ని $ తో వాడతాం

echo $name                # Yaswanth
echo "Name is $name"      # Name is Yaswanth (double quotes → variable expand
అవుతుంది)
echo 'Name is $name'      # Name is $name    (single quotes → literal, expand కాదు!)
echo "${name}_backup"     # ${} = boundary clear చేయడానికి → Yaswanth_backup

# command substitution – command output ని variable లోకి
```

```
current_dir=$(pwd)
file_count=$(ls | wc -l)
echo "Dir: $current_dir has $file_count files"

# arithmetic
result=$((count + 3))     # $(( )) = math
echo $result              # 8
```

Quoting rules – ఇవి పక్కాగా (interview + bugs):

Quote	ఏం చేస్తుంది	ఉదాహరణ
`"double"`	variables & `$(...)` **expand** అవుతాయి	`"Hello $name"` → `Hello Yaswanth`
`'single'`	అంతా **literal** (expand కాదు)	`'Hello $name'` → `Hello $name`
no quotes	expand + **word splitting** (⚠️ spaces తో bug)	`$file` (spaces ఉంటే విడిపోతుంది)

> ⚠️ **Golden rule:** variables ని ఎప్పుడూ **double quotes** లో వాడు – `"$file"`, `"$1"`. లేకపోతే filename లో space ఉంటే (`my file.txt`) command రెండు arguments గా చూస్తుంది → bug. ఇది shell scripting లో అత్యంత common mistake.

Arguments – `$1`, `$@`, `$#`

Script కి బయటి నుండి values పంపవచ్చు:

```
#!/bin/bash
# deploy.sh <environment> <version> గా call చేస్తాం
echo "Script name : $0"       # $0 = script పేరు (./deploy.sh)
echo "1st arg     : $1"       # $1 = మొదటి argument
echo "2nd arg     : $2"       # $2 = రెండవది
echo "All args    : $@"       # $@ = అన్ని arguments
echo "Arg count   : $#"       # $# = ఎన్ని arguments
echo "Exit status : $?"       # $? = చివరి command exit code
echo "This PID    : $$"       # $$ = ఈ script యొక్క PID
```

```
./deploy.sh production v2.3
# Script name : ./deploy.sh
# 1st arg     : production
# 2nd arg     : v2.3
# All args    : production v2.3
# Arg count   : 2
```

Conditions – `if`, `test`, `case`

```
#!/bin/bash
env="$1"

# if / elif / else
if [ "$env" = "production" ]; then
```

```
    echo "⚠️ Production deploy — జాగ్రత్త!"
elif [ "$env" = "staging" ]; then
    echo "Staging deploy"
else
    echo "Unknown env: $env"
fi

# file/number tests
if [ -f ".env" ]; then                  # -f = file exists
    echo ".env ఉంది"
fi
if [ ! -d "node_modules" ]; then        # -d = dir; ! = not
    echo "node_modules లేదు — npm install చేయాలి"
fi
if [ "$count" -gt 10 ]; then            # -gt = greater than (numbers)
    echo "count 10 కంటే ఎక్కువ"
fi
```

Test operators (`[ ]` లోపల):

Operator	అర్థం	ఉదాహరణ
`-f`	file ఉందా	`[ -f app.js ]`
`-d`	directory ఉందా	`[ -d src ]`
`-z`	string ఖాళీనా	`[ -z "$1" ]`
`-n`	string non-empty	`[ -n "$name" ]`
`=` / `!=`	string equal/not (strings)	`[ "$a" = "b" ]`
`-eq -ne -gt -lt -ge -le`	number compare	`[ "$n" -gt 5 ]`
`&&` `||`	AND / OR	`[ -f a ] && [ -f b ]`

⚠️ **Spacing:** `[ "$a" = "b" ]` — brackets చుట్టూ **spaces తప్పనిసరి** (`[` నిజానికి ఒక command). `["$a"="b"]` = error. Numbers కి `-eq/-gt`, strings కి `=`. Modern bash లో `[[ ... ]]` (double) safer — quoting bugs తక్కువ.

```
# case — multiple options (switch లాంటిది)
case "$1" in
    start)   echo "Starting..."; pm2 start app ;;
    stop)    echo "Stopping..."; pm2 stop app ;;
    restart) echo "Restarting..."; pm2 restart app ;;
    *)       echo "Usage: $0 {start|stop|restart}" ;;   # * = default
esac
```

Loops — `for`, `while`

```
#!/bin/bash
# for — list మీద iterate
for env in dev staging production; do
    echo "Deploying to $env"
done

# for — files మీద
for file in *.log; do
    echo "Compressing $file"
    gzip "$file"
done

# for — number range (C-style)
for ((i=1; i<=5; i++)); do
    echo "Attempt $i"
done

# while — condition ఉన్నంతవరకు
count=1
while [ "$count" -le 3 ]; do
    echo "Retry $count"
    count=$((count + 1))
done

# while — health check retry (real pattern):
until curl -sf http://localhost:3000/health > /dev/null; do
    echo "App start అవ్వడానికి wait..."
    sleep 2
done
echo "App ready! ✅"
```

Functions & exit codes

```
#!/bin/bash
# function define
log() {
    echo "[$(date +%H:%M:%S)] $1"     # $1 = function కి పంపిన 1st arg
}

check_command() {
    if ! command -v "$1" > /dev/null; then
        log "❌ $1 install కాలేదు"
        return 1                       # non-zero = failure
    fi
    log "✅ $1 ready"
    return 0                           # 0 = success
}

log "Deploy మొదలు"
check_command node
check_command npm

# exit codes: 0 = success, 1-255 = failure. $? = చివరి command exit code
npm run build
if [ $? -ne 0 ]; then                  # build fail అయితే
```

```
    log "Build failed! Aborting."
    exit 1                              # script ని failure code తో ఆపు
fi
```

Exit codes ముఖ్యం: ప్రతి command exit code ఇస్తుంది — `0` = success, non-zero = failure. `$?` తో చదువు. CI/CD (GitHub Actions, Jenkins) ఈ exit code చూసి "pipeline pass/fail" decide చేస్తుంది. అందుకే scripts లో fail అయితే `exit 1` తప్పనిసరి — లేకపోతే broken deploy కూడా "success" అనిపిస్తుంది.

`set` — safer scripts (SSE best practice)

```
#!/bin/bash
set -e            # ఏ command fail అయినా (non-zero), వెంటనే script ఆగిపో
set -u            # define చేయని variable వాడితే error (typo bugs పట్టుకుంటుంది)
set -o pipefail   # pipe లో ఏ భాగం fail అయినా మొత్తం fail
# కలిపి: set -euo pipefail    ← production scripts లో standard మొదటి line
```

`set -euo pipefail` ఎందుకు? Default గా bash ఒక command fail అయినా తర్వాతివి continue చేస్తుంది — deploy లో ప్రమాదకరం (build fail అయినా restart అవుతుంది). `set -e` = fail అయితే ఆగు. `set -u` = typo variable ($DATABSE_URL) పట్టుకుంటుంది. `pipefail` = pipe middle fail catch. ఈ line ప్రతి serious script మొదట్లో ఉండాలి.

⭐ Real useful script — MERN deploy

ఇప్పటివరకు నేర్చుకున్నవన్నీ కలిపి, ఒక actual deploy script:

```
#!/usr/bin/env bash
set -euo pipefail

# ---- MERN deploy script: ./deploy.sh <branch> ----
APP_DIR="/var/www/myapp"
BRANCH="${1:-main}"           # arg లేకపోతే default 'main' (:- = default value)
LOG_FILE="/var/log/deploy.log"

log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"; }

log "🚀 Deploy మొదలు — branch: $BRANCH"

cd "$APP_DIR" || { log "❌ $APP_DIR లేదు"; exit 1; }

log "📥 Latest code తెస్తున్నా..."
git fetch origin
git checkout "$BRANCH"
git pull origin "$BRANCH"

log "📦 Dependencies install..."
npm ci                        # ci = clean install (package-lock ప్రకారం,
faster/reliable)

log "🔨 Build..."
```

```
npm run build

log "🔄 App restart..."
pm2 restart myapp || pm2 start server.js --name myapp

log "🏥 Health check..."
sleep 3
if curl -sf http://localhost:3000/health > /dev/null; then
    log "✅ Deploy success! App healthy."
else
    log "❌ Health check fail! Logs చూడు: pm2 logs myapp"
    exit 1
fi
```

```
chmod +x deploy.sh
./deploy.sh main            # main branch deploy
./deploy.sh feature/login   # వేరే branch deploy
```

Key Points

- **Shebang** (`#!/usr/bin/env bash`) = ఏ interpreter అని OS కి చెప్తుంది. `chmod +x` తర్వాత `./script.sh`.
- **Variables:** `name=value` (space లేదు). Double quotes `"$var"` ఎప్పుడూ – space bugs తప్పించడానికి. Single quotes = literal.
- **Arguments:** `$1 $2` (positional), `$@` (అన్నీ), `$#` (count), `$?` (exit code), `$0` (script name).
- **Conditions:** `if [ ... ]` (spaces తప్పనిసరి), `-f/-d/-z`, numbers `-eq/-gt`. `case` = switch.
- **Loops:** `for x in list`, `while [ cond ]`, `until cond`. Functions: `name() { ... }`, `return N`.
- **`set -euo pipefail`** = production scripts safety. Exit code `0` =success – CI/CD దీన్ని చూస్తుంది.

Interview దృష్టి

Q: `set -e` ఎందుకు వాడతావ్? A: Default గా bash script లో ఒక command fail అయినా తర్వాతివి run అవుతూనే ఉంటాయి. Deploy script లో ఇది ప్రమాదం – `npm run build` fail అయినా app restart అవుతుంది (broken code deploy). `set -e` = ఏ command non-zero return చేసినా వెంటనే script ఆగి, failure code తో exit. Production scripts లో `set -euo pipefail` standard.

Q: `$@` మరియు `$*` తేడా? A: రెండూ అన్ని arguments. కానీ quotes లో: `"$@"` = ప్రతి argument ని separate గా ఉంచుతుంది (`"arg1" "arg2"` – spaces ఉన్న args safe), `"$*"` = అన్నిటినీ ఒకే string గా కలుపుతుంది. ఎప్పుడూ `"$@"` వాడు – arguments ని loop లో సరిగ్గా forward చేయడానికి.

Q: Variable ని double quotes లో ఎందుకు పెట్టాలి? A: Quotes లేకపోతే bash **word splitting** చేస్తుంది – variable లో space ఉంటే (filename `my file.txt`, లేదా `$@`) అది multiple arguments గా విడిపోతుంది → command తప్పు files మీద పనిచేస్తుంది లేదా fail అవుతుంది. `"$file"` గా quote చేస్తే ఒకే argument. Empty variable కూడా safe అవుతుంది (`[ -z "$x" ]`).

Part 3 — Git

> Git అనేది ప్రతి software team యొక్క **backbone**. కోడ్ ఎవరు, ఎప్పుడు, ఎందుకు మార్చారో track చేయడం; పాత versions కి తిరిగి వెళ్ళడం; ఇద్దరు devs ఒకే file మార్చినా conflict లేకుండా కలపడం — అంతా Git. Interview లో "git rebase అంటే ఏమిటి?", "conflict ఎలా resolve చేస్తావ్?", "accidentally main కి push చేశావ్ — undo ఎలా?" — ఇవి తప్పకుండా వస్తాయి. Self-taught developers చాలామంది `git add . && git commit -m "x" && git push` మాత్రమే వాడతారు — SSE అంటే branching, rebase, conflict resolution, reflog తో recover చేయడం confident గా రావాలి. ఈ Part లో Git ని mental model నుండి advanced recovery వరకు.

---

10. Git అంటే ఏమిటి, ఎందుకు — mental model

వివరణ

Git = distributed version control system (VCS). అంటే నీ code యొక్క ప్రతి version (snapshot) ని save చేసి, ఎప్పుడైనా వెనక్కి వెళ్ళగలిగే time-machine. "Distributed" = ప్రతి developer దగ్గర పూర్తి history యొక్క complete copy ఉంటుంది (central server మీద మాత్రమే ఆధారపడదు).

Version control ఎందుకు అవసరం? ఊహించు, Git లేకుండా:

- `project_final.zip`, `project_final_v2.zip`, `project_final_REALLY_final.zip` — ఇలా folders.
- "నిన్నటి working version ఏది?" గుర్తు లేదు.
- ఇద్దరు devs ఒకే file మార్చితే — ఒకరి మార్పులు మరొకరు overwrite.
- "ఈ bug ఎప్పుడు, ఎవరు introduce చేశారు?" తెలియదు.

Git ఇవన్నీ solve చేస్తుంది: ప్రతి change ఒక **commit** (labeled snapshot), full history, పాత version కి instant rollback, parallel work (branches), team merge.

git vs GitHub — ⚠️ చాలామంది confuse అవుతారు

అంశం	Git	GitHub
ఏమిటి	Tool/software (నీ computer లో)	Website/service (cloud లో host)
ఎవరిది	Open-source (Linus Torvalds, 2005)	Company (Microsoft owns)
పని	version control చేసే engine	git repos ని host చేసే platform
internet	అవసరం లేదు (local పనిచేస్తుంది)	అవసరం (cloud)

అంశం	Git	GitHub
alternatives	Mercurial, SVN	GitLab, Bitbucket
analogy	**camera** (photos తీసేది)	**Google Photos** (photos backup/share)

సారాంశం: Git = version control చేసే *tool*. GitHub = ఆ git repos ని online host చేసి, team collaboration (PRs, issues, CI) ఇచ్చే *platform*. Git లేకుండా GitHub లేదు; కానీ GitHub లేకుండా Git వాడవచ్చు (local, లేదా GitLab).

Real-life Scenario

> **Git = video game యొక్క save points.**
>
> Game ఆడేటప్పుడు ప్రతి important stage దగ్గర **save** చేస్తావు (commit). Boss fight లో చస్తే (bug/mistake), చివరి save point కి తిరిగి వెళ్ళి (git checkout/reset) మళ్ళీ ప్రయత్నిస్తావు – మొదటి నుండి కాదు. కావాలంటే ఒక save నుండి రెండు వేర్వేరు paths try చేయవచ్చు (branches) – ఒకటి నచ్చకపోతే వదిలేయవచ్చు.
>
> **GitHub = ఆ save files ని cloud లో backup + friends తో share చేయడం.** నీ laptop పోయినా, cloud నుండి తెచ్చుకోవచ్చు. Friends కూడా అదే game లో collaborate చేయవచ్చు (team). Git లేకుండా = save points లేని game – చస్తే మొదటి నుండి. అందుకే ఏ developer కూడా Git లేకుండా పని చేయడు.

Git ఎలా store చేస్తుంది – snapshots, deltas కాదు

చాలా VCS లు (SVN) "ఏం మారింది" (deltas/differences) మాత్రమే store చేస్తాయి. **Git వేరే** – ప్రతి commit కి file system యొక్క **పూర్తి snapshot** తీస్తుంది (మారని files కి పాత snapshot కి pointer వాడి efficient గా). అందుకే branch switch, history navigation చాలా fast.

ప్రతి commit కి unique **SHA-1 hash** (ఉదా `a3f5c9e...`) – ఇది ఆ snapshot యొక్క content + parent + author + time అన్నిటి fingerprint. అందుకే history ని ఎవరూ silent గా మార్చలేరు (మార్చితే hash మారుతుంది).

⭐ మూడు areas – ఇది Git కి గుండె (interview must-know)

Git లో ఒక file ఒక్కసారిగా repo లోకి వెళ్ళదు – **మూడు areas** గుండా వెళ్తుంది:

```
Working Directory        Staging Area (Index)        Repository (.git)
(నీ actual files)       (commit కి "ready" చేసినవి)    (permanent history)
       |                          |                            |
       |   git add file           |        git commit          |
       |  ----------------------> |  ------------------------> |
       |                          |                            |
       |<------------------------------------------------------|
       |              git checkout / git restore               |
```

Area	ఏమిటి	ఎలా చేరతాయి
Working Directory	నీ project folder – నువ్వు నిజంగా edit చేసే files	file edit చేస్తే
Staging Area (Index)	తర్వాతి commit లో ఏం వెళ్ళాలో "ఎంపిక" చేసిన snapshot	`git add`
Repository (.git)	permanent, committed history (అన్ని snapshots)	`git commit`

ఎందుకు staging area? ఇది Git యొక్క unique feature. నువ్వు 5 files మార్చావు, కానీ 2 మాత్రమే ఒక logical commit కి సంబంధించినవి. `git add file1 file2` తో ఆ 2 మాత్రమే stage చేసి commit చేయవచ్చు – మిగతా 3 తర్వాత separate commit. అంటే staging = "ఈ commit లో ఏం పెట్టాలో ముందు review/select చేసుకునే బెంచ్".

Real-life Scenario (3 areas)

> **మూడు areas = ఒక courier పంపే process.**
>
> - **Working Directory** = నీ ఇల్లు – ఇక్కడ వస్తువులు (files) ఇష్టంగా అటుఇటు మారుస్తావు.
> - **Staging Area** = courier office లో packing table – ఏ వస్తువులు ఈ box లో పంపాలో ముందు table మీద పెట్టి, check చేసి, pack చేస్తావు. అన్నీ కాదు, ఎంచుకున్నవే.
> - **Repository** = courier నిజంగా pickup చేసి, permanent tracking number (commit hash) ఇచ్చి, పంపేసిన స్థితి. ఇక వెనక్కి తీయలేవు (కానీ record ఉంటుంది).
>
> నువ్వు `git add` = table మీద పెట్టడం (staging). `git commit` = courier pickup + tracking number. అందుకే add చేయకుండా commit చేస్తే "ఏమీ pack చేయలేదు" (nothing to commit).

Commands – mental model verify

```
git --version                # git ఉందా, ఏ version
# git version 2.43.0

# మొదటిసారి setup (identity — ప్రతి commit కి పేరు attach అవుతుంది):
git config --global user.name "Yaswanth"
git config --global user.email "you@email.com"
git config --global init.defaultBranch main   # default branch పేరు main
git config --list                              # అన్ని settings చూడు

# ఒక file యొక్క ప్రయాణం చూద్దాం:
echo "hello" > app.js
git status
# Untracked files:  app.js       ← working dir లో ఉంది, git కి తెలియదు

git add app.js
git status
# Changes to be committed:  new file: app.js   ← staging లోకి వచ్చింది

git commit -m "Add app.js"
git status
# nothing to commit, working tree clean      ← repo లోకి వెళ్ళింది
```

Key Points

- **Git = distributed version control tool** (local); **GitHub = cloud platform** git repos ని host చేసేది. కెమెరా vs Google Photos.
- Git **snapshots** store చేస్తుంది (deltas కాదు); ప్రతి commit కి unique **SHA hash** = tamper-proof history.
- ⭐ **మూడు areas: Working Directory → (`git add`) → Staging → (`git commit`) → Repository.** ఇది Git కి foundation.
- **Staging area** = commit లో ఏం పెట్టాలో ముందు select చేసుకునే బెంచ్ – Git యొక్క unique feature.
- మొదటిసారి `git config --global user.name/email` set చేయాలి (ప్రతి commit కి attach).

Interview దృష్టి

Q: Git మరియు GitHub తేడా? A: Git = version control చేసే *tool*, నీ machine లో local గా పనిచేస్తుంది (internet అవసరం లేదు). GitHub = ఆ git repositories ని cloud లో host చేసే *platform* – team collaboration (pull requests, code review, issues, CI/CD) ఇస్తుంది. GitLab, Bitbucket = GitHub alternatives. Git లేకుండా GitHub ఉండదు.

Q: Staging area ఎందుకు ఉంది, దాని ప్రయోజనం? A: Working directory, repository మధ్య intermediate layer. ఇది **selective commits** ని enable చేస్తుంది – నువ్వు 10 files మార్చినా, ఒక logical unit కి సంబంధించిన files మాత్రమే `git add` చేసి commit చేయవచ్చు. Commit చేసే ముందు "ఏం వెళ్తోంది?" review చేసుకోవచ్చు (`git diff --staged`). ఇది clean, atomic commit history కి సహాయం.

Q: Git snapshots vs deltas – ఎందుకు ముఖ్యం? A: Git ప్రతి commit కి full snapshot తీస్తుంది (మారని files కి previous snapshot కి pointer). SVN లాంటివి deltas (changes మాత్రమే) store చేస్తాయి. Snapshots వల్ల branch switching, history navigation, checkout అన్నీ చాలా fast – ఎందుకంటే Git ఏ commit నైనా directly reconstruct చేయగలదు, deltas ని వరుసగా apply చేయనవసరం లేదు.

---

11. Git Basics – init, clone, add, commit, status, log, diff

GIT · WORKING DIR, STAGING, REPO

`git add -p`).దీంతో commits చిన్నగా, తార్కికంగా, review చేయదగినవిగా ఉంటాయి.**వెనక్కి తీసుకోవడం – ఏది దేన్ని తాకుతుంది** `git restore <file>` – working dir · `git restore --staged` – staging `git reset --soft` – commit మాత్రం · `--mixed` – + staging · `--hard` – + working dir (ప్రమాదకరం)

వివరణ

ఇవి Git లో నీ రోజువారీ 90% commands. కొత్త project మొదలుపెట్టడం (`init`), ఉన్నదాన్ని తెచ్చుకోవడం (`clone`), మార్పులు record చేయడం (`add` → `commit`), ఏం జరిగిందో చూడటం (`status` , `log` , `diff`), వద్దనుకున్న files ని ignore చేయడం (`.gitignore`). ఒక్కొక్కటి flags సహా చూద్దాం.

Real-life Scenario

> **Git basics = ఒక diary రాయడం.**
>
> - `git init` = కొత్త diary కొనడం.
> - `git clone` = ఇంకొకరి diary యొక్క complete copy తీసుకోవడం.
> - `git add` = ఇవాళ ఏం రాయాలో draft లో ఎంచుకోవడం.
> - `git commit -m "..."` = ఆ page ని date + note తో permanent గా diary లో రాయడం.
> - `git log` = గత pages అన్నీ తిరగేసి చూడటం.
> - `git diff` = draft లో ఏం మారిందో పోల్చి చూడటం.
>
> ప్రతి commit ఒక diary entry – date, author, message తో. తర్వాత "ఆ రోజు ఏం జరిగింది?" అని log చూసి తెలుసుకోవచ్చు. మంచి commit message = మంచి diary entry ("Fixed login bug" not "asdf").

Repository start: `init` & `clone`

```
# --- init: కొత్త repo (existing folder లో) ---
mkdir myapp && cd myapp
git init                        # .git folder create → ఇప్పుడు git tracked
# Initialized empty Git repository in /home/ubuntu/myapp/.git/
ls -a                           # .git కనిపిస్తుంది (అంతా ఇక్కడే stored)

# --- clone: existing repo ని copy ---
git clone https://github.com/user/repo.git       # HTTPS
git clone git@github.com:user/repo.git           # SSH (key setup ఉంటే)
git clone https://github.com/user/repo.git myapp # వేరే folder పేరుతో
git clone --depth 1 https://github.com/user/repo.git  # shallow (చివరి commit మాత్రమే —
fast, CI కి)
cd repo                         # clone folder లోకి
```

> `.git` **folder = మొత్తం Git.** అది delete చేస్తే – files ఉంటాయి కానీ history అంతా పోతుంది (ordinary folder అవుతుంది). Clone చేసినప్పుడు ఈ `.git` (full history) కూడా వస్తుంది – అందుకే "distributed" (ప్రతి clone దగ్గర complete history).

Record changes: `add` & `commit`

```
git add app.js                  # ఒక file stage
git add src/                    # ఒక folder మొత్తం
git add .                       # అన్ని మార్పులు stage (⚠️ .gitignore honor చేస్తుంది)
git add -A                      # అన్నీ (deletions కూడా)
git add -p                      # -p = patch mode — ఒక file లో కొన్ని భాగాలే stage
```

```
(interactive)

git commit -m "Add login API"  # stage అయినవి commit (message తో)
git commit -am "Fix typo"      # -a = tracked files ని auto-add + commit (ఒకే step;
untracked కి పనిచేయదు)
git commit                     # -m లేకపోతే editor తెరుస్తుంది (long message కి)

# commit message convention (SSE లో మంచి practice):
git commit -m "feat: add JWT authentication to /login endpoint"
# feat/fix/docs/refactor/test/chore: <short description>
```

మంచి commit messages:

❌ చెడు	✅ మంచి
`"changes"`	`"fix: prevent null crash in user profile"`
`"asdf"`	`"feat: add pagination to products API"`
`"final fix"`	`"refactor: extract auth logic into middleware"`

> **Convention (Conventional Commits):** `type: description` — `feat` (కొత్త feature), `fix` (bug fix), `docs`, `refactor`, `test`, `chore` (build/deps). ఇది automated changelogs, semantic versioning కి సహాయం. చాలా teams దీన్ని enforce చేస్తాయి.

Inspect: `status`, `log`, `diff`

```
# --- status: ఇప్పుడు ఏ state లో ఉన్నాం ---
git status
# On branch main
# Changes not staged for commit:  modified: app.js   (working dir లో మారింది, add
కాలేదు)
# Untracked files: new.js                             (git కి కొత్తది)
git status -s                  # -s = short format (compact)
#  M app.js       (M=modified, A=added, ?? =untracked, D=deleted)
# ?? new.js

# --- log: commit history ---
git log
# commit a3f5c9e... (HEAD -> main)
# Author: Yaswanth <you@email.com>
# Date:   Thu Jul 17 ...
#     feat: add login
git log --oneline              # ⭐ compact (ఒక commit = ఒక line)
# a3f5c9e feat: add login
# b2d4a1f init commit
git log --oneline --graph --all  # branches ని visual graph గా (చాలా useful)
git log -5                     # చివరి 5 commits
git log --author="Yaswanth"    # ఒక author commits
git log --oneline app.js       # ఒక file యొక్క history
git log -p app.js              # ఆ file కి ప్రతి commit లో ఏం మారిందో

# --- diff: ఏం మారింది (line-by-line) ---
```

```
git diff                    # working dir vs staging (add చేయని మార్పులు)
git diff --staged           # staging vs last commit (commit అవ్వబోయేవి)
git diff HEAD               # working + staging vs last commit (అన్ని మార్పులు)
git diff main feature       # రెండు branches మధ్య
git diff a3f5c9e b2d4a1f    # రెండు commits మధ్య
git show a3f5c9e            # ఒక commit లో ఏం మారిందో పూర్తిగా
```

diff యొక్క రెండు దశలు (గందరగోళం clear):

Command	ఏం compare చేస్తుంది
`git diff`	Working directory ↔ Staging (**ఇంకా `add` చేయని** మార్పులు)
`git diff --staged`	Staging ↔ Last commit (`add` **చేసి commit అవ్వబోయే** మార్పులు)
`git diff HEAD`	Working+Staging ↔ Last commit (అన్నీ కలిపి)

`.gitignore` – ఏం track చేయకూడదో

కొన్ని files git లో ఉండకూడదు: dependencies (`node_modules` – పెద్దది, `npm install` చేసుకోవచ్చు), secrets (`.env`), build output (`dist/`), logs, OS files. వీటిని `.gitignore` లో list చేస్తే git ignore చేస్తుంది.

```
# .gitignore (project root లో) — MERN typical:
node_modules/          # dependencies — ఎప్పుడూ commit చేయకు (పెద్దది, regenerate)
.env                   # secrets — ఎప్పుడూ commit చేయకు!
.env.local
dist/                  # build output
build/
*.log                  # అన్ని log files
npm-debug.log*
coverage/              # test coverage
.DS_Store              # macOS junk
.vscode/               # editor settings (కొన్ని teams commit చేస్తాయి)
*.tmp
```

```
# .gitignore patterns:
node_modules/     # folder (ఎక్కడైనా)
*.log             # extension
/config.js        # root లో మాత్రమే (subfolder కాదు)
!important.log    # ! = ఈ file ని ignore చేయకు (exception)
temp/*.txt        # temp లో .txt లు
```

⚠️ **అత్యంత common mistake:** file ని **commit చేశాక** `.gitignore` లో పెడితే పనిచేయదు – `.gitignore` కేవలం **untracked** files కి applies. ఇప్పటికే tracked file ని stop చేయాలంటే: `git rm --cached <file>` (working dir లో ఉంచి, git tracking నుండి తీసేయి), తర్వాత commit. `.env` accidentally commit అయితే – history నుండి తీసేసి, secret rotate చేయాలి (leaked).

```
# ఇప్పటికే tracked file ని ignore చేయడం:
git rm --cached .env          # git tracking నుండి తీసేయి (file disk లో ఉంటుంది)
echo ".env" >> .gitignore     # gitignore కి కలుపు
git commit -m "chore: stop tracking .env"
```

Key Points

- **`git init`** = కొత్త repo; **`git clone`** = ఉన్నదాన్ని full history తో copy. `.git` folder = మొత్తం Git.
- **`git add`** (stage) → **`git commit -m`** (record). `git add .` = అన్నీ, `git add -p` = selective.
- మంచి commit messages: **`type: description`** (feat/fix/refactor). History = documentation.
- **`git status`** (ఇప్పటి state), **`git log --oneline`** (history), **`git diff`** (unstaged) vs **`git diff --staged`** (staged).
- **`.gitignore`** = node_modules/.env/dist ని track చేయకు. కేవలం **untracked** files కి; already-tracked కి `git rm --cached`.

Interview దృష్టి

Q: `git add .` మరియు `git add -A` తేడా? A: ఆధునిక Git లో దాదాపు ఒకటే – రెండూ అన్ని మార్పులు (new, modified, deleted) stage చేస్తాయి. చారిత్రకంగా `git add .` current directory + subdirs మాత్రమే చూసేది, `-A` whole repo. ఇద్దరూ `.gitignore` honor చేస్తారు. `git add -p` = interactive, ఒక file లో కొన్ని hunks మాత్రమే stage (clean commits కి).

Q: `node_modules` ని ఎందుకు commit చేయకూడదు? A: (1) చాలా పెద్దది (వేల files, MB/GB) – repo bloat. (2) `package.json` + `package-lock.json` నుండి `npm ci` తో exactly regenerate అవుతుంది. (3) platform-specific binaries ఉంటాయి (nlicense mac vs Linux). అందుకే `.gitignore` లో పెట్టి, lockfile commit చేసి, deploy లో `npm ci`. ఇది Git best practice.

Q: Accidentally `.env` commit అయ్యింది – ఏం చేస్తావ్? A: (1) `git rm --cached .env` + `.gitignore` కి add + commit – future commits లో ఉండదు. (2) కానీ **history లో ఇంకా ఉంది** – ఎవరైనా పాత commit నుండి చదవగలరు. అందుకే leaked secrets ని **rotate** చేయాలి (కొత్త keys generate). (3) పూర్తిగా history నుండి తీయాలంటే `git filter-repo` లేదా BFG tool (force-push అవసరం, team ని alert చేయాలి).

---

12. Branching & Merging – branch, switch, merge, HEAD

వివరణ

Branch = code యొక్క ఒక independent line of development. `main` (production-ready code) ని ముట్టకుండా, కొత్త feature ని ఒక separate branch లో develop చేస్తావు. అయిపోయాక `main` లోకి **merge** చేస్తావు. అంటే multiple developers ఒకే codebase లో, ఒకరికొకరు అడ్డు రాకుండా parallel గా పని చేయగలరు. ఇదే Git యొక్క killer feature – branches చాలా cheap, fast (కేవలం ఒక pointer).

Real-life Scenario

> **Branch = ఒక document యొక్క "draft copy".**
>
> నీ team ఒక పుస్తకం రాస్తోంది (`main` = published version). నువ్వు "Chapter 5 ని మార్చాలి" అనుకుంటే – published version ని నేరుగా ముట్టవు (readers చదువుతున్నారు). బదులుగా ఒక draft copy (branch) తీసుకుని, అందులో మార్పులు చేస్తావు. అదే సమయంలో నీ colleague మరో draft లో Chapter 8 మారుస్తాడు. మీ ఇద్దరి draft లు ready అయ్యాక, editor వాటిని published version లోకి **merge** చేస్తాడు. ఒకరి పని మరొకరికి అడ్డు రాదు.
>
> **HEAD = "నేను ఇప్పుడు ఏ draft లో పని చేస్తున్నాను?" అని చూపించే bookmark.** Branch switch చేస్తే HEAD ఆ branch కి కదులుతుంది. అందుకే commit చేస్తే అది HEAD ఉన్న branch కి వెళ్తుంది.

`HEAD` – నువ్వు ఎక్కడ ఉన్నావో pointer

- **HEAD** = "ప్రస్తుతం నువ్వు ఏ commit/branch మీద ఉన్నావో" చూపించే pointer.
- సాధారణంగా HEAD → ఒక branch (`main`) → ఆ branch యొక్క చివరి commit.
- `git commit` చేస్తే – HEAD ఉన్న branch ముందుకు కదులుతుంది.
- **detached HEAD** = HEAD నేరుగా ఒక commit కి point చేస్తుంది (branch కి కాదు) – ఉదా `git checkout <hash>`. అక్కడ commit చేస్తే ఏ branch కి attach అవ్వదు (దాదాపు lost – జాగ్రత్త).

Branch commands: `branch`, `switch`, `checkout`

```
git branch                     # అన్ని local branches (* = ప్రస్తుతం ఉన్నది)
# * main
#   feature/login
git branch -a                  # remote branches కూడా
git branch feature/login       # కొత్త branch create (కానీ switch అవ్వదు)

# --- switch (ఆధునిక, clear – branch switching కి prefer) ---
git switch feature/login       # ఆ branch కి మారు
git switch -c feature/signup   # -c = create + switch (ఒకే step) ⭐ common
git switch main                # main కి తిరిగి
git switch -                   # ముందు ఉన్న branch కి (cd - లాంటిది)

# --- checkout (పాత, multipurpose – branch + files + commits) ---
git checkout feature/login     # branch switch (switch కి పాత రూపం)
git checkout -b feature/new     # create + switch (switch -c కి సమానం)
git checkout a3f5c9e            # ఒక commit కి వెళ్ళు (⚠️ detached HEAD)

# branch delete
git branch -d feature/login    # -d = delete (merge అయితేనే – safe)
git branch -D feature/login    # -D = force delete (merge కాకపోయినా)
git branch -m old-name new-name  # rename
```

> `switch` vs `checkout` **(interview లో అడుగుతారు):** పాత రోజుల్లో `checkout` అన్నిటికీ – branch మార్చడం, files restore చేయడం, commits కి వెళ్ళడం (overloaded, confusing). Git 2.23 (2019) లో దీన్ని విడగొట్టారు: **`git switch`** = branches కి మాత్రమే, **`git restore`** = files restore కి. కొత్త code లో `switch` / `restore` prefer చేయి (clearer intent), కానీ `checkout` ఇంకా పనిచేస్తుంది (legacy scripts, tutorials).

Merging – branches ని కలపడం

Feature branch పని అయ్యాక, దాన్ని `main` లోకి merge చేస్తావు:

```
git switch main                # ముందు target branch కి వెళ్ళు (ఎక్కడికి merge చేస్తున్నామో)
git merge feature/login        # feature/login ని main లోకి తెచ్చు
# Updating a3f5c9e..b2d4a1f
# Fast-forward
#  login.js | 20 ++++++++++
```

రెండు రకాల merges:

1. Fast-forward merge – main branch, నువ్వు branch తీసినప్పటి నుండి మారకపోతే:

```
merge ముందు:
main:     A──B
              \
feature:       C──D   (main కి కొత్త commits లేవు)

git merge feature తర్వాత (fast-forward):
main:     A──B──C──D    (main కేవలం ముందుకు జరిగింది – కొత్త commit లేదు)
```

main pointer ని feature చివరికి "జరిపేయడం" మాత్రమే. కొత్త merge commit లేదు. History linear గా ఉంటుంది.

2. Three-way merge – నువ్వు branch తీశాక main కూడా మారితే (divergent):

```
merge ముందు:
main:     A──B──E──F     (main లో కొత్త commits E, F వచ్చాయి)
              \
feature:       C──D

git merge feature తర్వాత (3-way):
main:     A──B──E──F──M   ← M = merge commit (2 parents: F మరియు D)
              \        /
feature:       C──D──┘
```

Git రెండు branches ని కలిపి కొత్త **merge commit (M)** create చేస్తుంది – దీనికి **two parents** (F, D). History లో "ఇక్కడ branch merge అయ్యింది" కనిపిస్తుంది.

Fast-forward vs 3-way:

అంశం	Fast-forward	Three-way
ఎప్పుడు	target branch మారలేదు	రెండు branches diverge అయ్యాయి
merge commit	❌ లేదు	✅ ఒకటి create అవుతుంది (2 parents)
history	linear (clean)	branch structure కనిపిస్తుంది
force చేయడం	`--no-ff` (ff possible అయినా merge commit)	automatic

```
git merge --no-ff feature/login    # fast-forward possible అయినా merge commit create
#   (feature ఎక్కడ merge అయ్యిందో history లో కనిపించాలని teams దీన్ని వాడతాయి)
git merge --squash feature/login   # feature యొక్క అన్ని commits ని ఒక్క commit గా (history
clean)
```

> `--no-ff` **ఎందుకు?** Fast-forward లో feature branch history లో "మిళితమైపోతుంది" – ఏ commits ఒక feature కి చెందినవో తెలియదు. `--no-ff` ఎప్పుడూ merge commit create చేస్తుంది → "ఈ feature ఇక్కడ merge అయ్యింది" clear గా కనిపిస్తుంది. GitLab/GitHub PR merges default గా ఇలా చేస్తాయి.

Complete workflow ఉదాహరణ

```
# 1. కొత్త feature కి branch
git switch -c feature/user-profile

# 2. పని చేయి, commits చేయి
echo "profile code" > profile.js
git add profile.js
git commit -m "feat: add user profile page"

# 3. మధ్యలో main లో ఎవరో మార్చారు అనుకో – update తెచ్చుకో
git switch main
git pull origin main          # latest main
git switch feature/user-profile
git merge main                # main మార్పులని feature లోకి తెచ్చు (conflicts ముందే
resolve)

# 4. feature ready – main లోకి merge
git switch main
git merge feature/user-profile
git push origin main

# 5. cleanup
git branch -d feature/user-profile
```

Key Points

- **Branch = independent line of development.** `main` ని ముట్టకుండా features develop చేయడానికి. చాలా cheap (కేవలం pointer).

- `git switch -c name` = create + switch (ఆధునికం); `git branch -d` = delete. `checkout` = పాత multipurpose.
- **HEAD** = నువ్వు ఇప్పుడు ఏ commit/branch మీద ఉన్నావో pointer. detached HEAD = branch కి కాదు, commit కి direct.
- **Fast-forward** = target మారకపోతే pointer ముందుకు (linear, no merge commit). **3-way** = diverge అయితే merge commit (2 parents).
- `--no-ff` = ఎప్పుడూ merge commit (feature boundary కనిపించడానికి); `--squash` = అన్నిటినీ ఒక commit గా.

Interview దృష్టి

Q: Fast-forward merge అంటే ఏమిటి, ఎప్పుడు జరుగుతుంది? A: నువ్వు branch తీసిన తర్వాత target branch (main) లో కొత్త commits ఏమీ రాకపోతే – Git కొత్త merge commit create చేయకుండా, కేవలం main pointer ని feature branch చివరికి "ముందుకు జరిపేస్తుంది" (fast-forward). History linear గా ఉంటుంది. Diverge అయితే (main కూడా మారితే) fast-forward సాధ్యం కాదు – 3-way merge (merge commit) అవుతుంది.

Q: `git switch` ఎందుకు introduce చేశారు, `checkout` ఉండగా? A: `checkout` overloaded – branches మార్చడం, files restore చేయడం, commits కి వెళ్ళడం అన్నీ ఒకే command. Confusing + dangerous (`git checkout file` accidentally మార్పులు పోగొట్టగలదు). Git 2.23 లో దీన్ని `switch` (branches) + `restore` (files) గా విడగొట్టారు – intent clear, safer. Checkout backward-compat కి ఉంది.

Q: `--no-ff` ఎందుకు వాడతారు? A: Fast-forward merge లో feature commits linear history లో కలిసిపోతాయి – "ఏ commits ఒక feature కి చెందినవి?" తెలియదు. `--no-ff` ఎప్పుడూ merge commit create చేస్తుంది, ఫలితంగా feature branch boundary history లో స్పష్టంగా కనిపిస్తుంది. Revert చేయాలంటే మొత్తం feature ని ఒక్క merge commit revert తో వెనక్కి తీయవచ్చు. అందుకే చాలా teams దీన్ని standard చేస్తాయి.

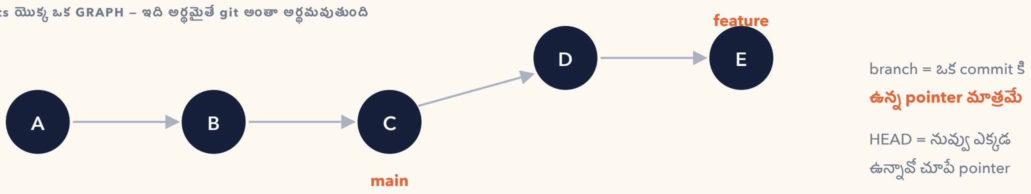
---

13. Rebase vs Merge + Conflicts resolve చేయడం

వివరణ

Feature branch ని up-to-date చేయడానికి, లేదా main లోకి integrate చేయడానికి **రెండు మార్గాలు**: `merge` (branches ని కలిపి merge commit create) లేదా `rebase` (నీ commits ని target branch చివర "మళ్ళీ apply" చేయడం). ఇది interview లో అత్యంత అడిగే git topic. తర్వాత – ఇద్దరు ఒకే line మార్చినప్పుడు వచ్చే **conflict** ని ఎలా resolve చేయాలో step-by-step.

GIT = commits యొక్క ఒక GRAPH – ఇది అర్థమైతే git అంతా అర్థమవుతుంది



MERGE

రెండు చరిత్రలని కలిపి ఒక కొత్త commit
చరిత్ర నిజాయితీగా ఉంటుంది (branching కనిపిస్తుంది)
Shared branches కి ఇదే సురక్షితం

REBASE

నా commits ని కొత్త base మీద మళ్ళీ రాయడం
చరిత్ర సరళరేఖగా, శుభ్రంగా ఉంటుంది
⚠ push అయిన commits ని ఎప్పుడూ rebase చేయొద్దు

బంగారు నియమం

నీ సొంత local branch – rebase చేసుకో, చరిత్ర శుభ్రంగా ఉంటుంది.
ఇతరులు కూడా వాడుతున్న branch – merge మాత్రమే. Rebase వాళ్ళ చరిత్రని విరగ్గొడుతుంది.

Real-life Scenario

> **Merge vs Rebase = ఒక Google Doc లో మీ మార్పులని team doc తో sync చేయడం.**
>
> నువ్వు Chapter 5 draft మీద పని చేశావు. అదే సమయంలో team main doc కి కొత్త chapters కలిపారు. ఇప్పుడు నీ మార్పులని team doc తో కలపాలి –
>
> - **Merge** = "ఇక్కడ నా draft ని team version తో కలిపాను" అని ఒక note (merge commit) పెట్టి రెండింటినీ join చేయడం. History లో "కలిసిన చోటు" కనిపిస్తుంది – నిజం, కానీ కాస్త messy.
> - **Rebase** = నీ మార్పులని తీసేసి, latest team doc ని base గా తీసుకుని, నీ మార్పులని దాని పైన మళ్ళీ రాయడం. ఫలితం – నువ్వు మొదటి నుండి latest version మీదే పని చేసినట్టు, clean linear story. కానీ "అసలు history" ని తిరగరాశావు.
>
> **Conflict** = నువ్వూ, colleague ఇద్దరూ అదే వాక్యాన్ని వేర్వేరుగా మార్చారు – Git "ఏది ఉంచాలో నాకు తెలియదు, నువ్వే decide చెయ్" అంటుంది. అప్పుడు నువ్వు manually ఏది సరైనదో ఎంచుకుంటావు.

Merge vs Rebase – visual

అదే starting point:

```
main:     A——B——E——F      (E, F = నువ్వు branch తీశాక వచ్చిన main commits)
              \
feature:       C——D         (నీ feature commits)
```

`git merge main` **(feature లో ఉండి):**

```
feature:  A——B——C——D——M   ← merge commit M (2 parents: D, F)
              \        /
main:          E——F——┘
```

నీ commits (C, D) అలాగే ఉంటాయి; కొత్త merge commit M వస్తుంది. History branchy గా ఉంటుంది కానీ **నిజం** (ఏం జరిగిందో అలాగే).

`git rebase main` (feature లో ఉండి):

```
main:     A——B——E——F
                     \
feature:              C'——D'   ← C, D లని F పైన మళ్ళీ apply (కొత్త hashes C', D')
```

నీ commits ని తీసేసి, latest main (F) పైన మళ్ళీ apply చేస్తుంది → **linear** history (branch తీసిన గుర్తు లేదు). C', D' = కొత్త commits (content అదే కానీ hash మారింది – rewritten).

Merge vs Rebase – comparison

అంశం	Merge	Rebase
history	branchy (merge commits)	linear (clean, straight line)
commits	original అలాగే	rewritten (కొత్త hashes)
merge commit	✅ create అవుతుంది	❌ లేదు
నిజం vs clean	actual history (నిజం)	idealized (clean కానీ rewritten)
conflicts	ఒకేసారి resolve	ప్రతి commit కి రావచ్చు
ఎప్పుడు	shared/public branches	నీ local private branch
⚠️ danger	safe	pushed commits ని rebase = ప్రమాదం

⭐ Golden Rule – ఎప్పుడు ఏది

> **"Rebase your local, private branches. Merge public, shared branches."**
>
> - **Rebase** = నీ **local feature branch** ని latest main తో up-to-date చేయడానికి (push ముందు). Clean linear history.
> - **Merge** = feature ని **main (shared) లోకి** integrate చేయడానికి. లేదా already-pushed branch కి.
> - ⚠️ **NEVER rebase a branch that others have pulled/pushed.** Rebase commit hashes మారుస్తుంది – team members యొక్క history తో diverge అవుతుంది, గందరగోళం. "Public history ని rewrite చేయకు" = Git యొక్క first rule.

```
# feature branch ని latest main తో update (rebase – clean):
git switch feature/login
git fetch origin
git rebase origin/main          # నీ commits ని latest main పైన apply
# (conflicts వస్తే resolve → git rebase --continue)
```

```
git push --force-with-lease    # rebase తర్వాత push కి force అవసరం (history మారింది)
#       --force-with-lease = safer force (ఇంకెవరో push చేసి ఉంటే fail అవుతుంది)

# --- interactive rebase: commits ని cleanup (squash, reword, reorder) ---
git rebase -i HEAD~3           # చివరి 3 commits ని edit
#   editor లో: pick/squash/reword/drop
#   pick   a3f5c9e feat: add login
#   squash b2d4a1f fix typo         ← ముందు commit లోకి కలుపు
#   reword c1e8f2a wip              ← message మార్చు
#   (WIP/typo commits ని ఒక clean commit గా చేయడానికి — PR ముందు)
```

> `--force-with-lease` **vs** `--force`**:** rebase/amend history మారుస్తుంది కాబట్టి normal push reject అవుతుంది. `--force` గుడ్డిగా overwrite చేస్తుంది (ఇంకెవరో push చేసి ఉంటే వారి పని పోతుంది). `--force-with-lease` = remote నువ్వు చివరిసారి చూసినట్టే ఉంటేనే push (లేకపోతే fail) — safer. ఎప్పుడూ `--force-with-lease`.

⭐ Conflicts — step-by-step resolution

Conflict = ఇద్దరు **అదే file, అదే lines** వేర్వేరుగా మార్చారు. Git ఏది ఉంచాలో తెలియక నిన్ను అడుగుతుంది.

Step 1 — conflict జరుగుతుంది:

```
git merge feature/login
# Auto-merging config.js
# CONFLICT (content): Merge conflict in config.js
# Automatic merge failed; fix conflicts and then commit the result.

git status
# Unmerged paths:
#   both modified:   config.js      ← ఈ file లో conflict
```

Step 2 — conflict markers చూడు. Git conflicted file లో ఇలా markers పెడుతుంది:

```
const config = {
<<<<<<< HEAD
  port: 3000,           // నీ branch (current) version
  timeout: 5000,
=======
  port: 8080,           // వచ్చే branch (incoming) version
  timeout: 3000,
>>>>>>> feature/login
};
```

- `<<<<<<< HEAD` నుండి `=======` వరకు = **నీ version** (current branch).
- `=======` నుండి `>>>>>>>` వరకు = **incoming version** (merge చేస్తున్న branch).

Step 3 — manually resolve. Markers తీసేసి, సరైన final version రాయి (ఒకటి, రెండూ, లేదా mix):

```
const config = {
  port: 8080,           // incoming ఉంచాలని decide చేశాను
```

```
  timeout: 5000,       // నా version ఉంచాను (రకలిపాను)
};
```

అన్ని `<<<<<<<`, `=======`, `>>>>>>>` markers తీసేయాలి (వదిలేస్తే syntax error).

Step 4 – resolved అని mark చేసి, పూర్తి చెయ్:

```
git add config.js              # resolved అని Git కి చెప్పు
git status                     # అన్ని conflicts resolve అయ్యాయో చూడు
git commit                     # merge పూర్తి (message auto – "Merge branch...")
#   (rebase అయితే: git rebase --continue)
```

Rebase conflict లో తేడా: rebase ప్రతి commit ని ఒక్కొక్కటిగా apply చేస్తుంది కాబట్టి, ఒకే conflict multiple commits లో రావచ్చు:

```
git rebase origin/main
# CONFLICT in config.js
# ... resolve (Step 2-3 లాగే) ...
git add config.js
git rebase --continue          # commit కాదు – continue (Git message reuse చేస్తుంది)
# ... తర్వాతి commit లో మళ్ళీ conflict రావచ్చు → repeat ...

git rebase --abort             # ⭐ గందరగోళం అయితే – మొత్తం cancel, rebase ముందటి స్థితికి
git merge --abort              # merge కి కూడా – conflict వదిలేసి వెనక్కి
```

Conflict resolution tools:

```
git checkout --ours config.js    # conflict లో పూర్తిగా నా version ఉంచు
git checkout --theirs config.js  # పూర్తిగా incoming version ఉంచు
git mergetool                    # visual merge tool తెరుస్తుంది (VS Code, meld...)
git diff                         # conflict markers ఉన్న chunks చూడు
```

> ⚠️ **Rebase లో ours/theirs తారుమారు:** rebase సమయంలో "ours" = మీరు rebase చేస్తున్న *target* (main), "theirs" = మీ feature commits – merge కి opposite! ఎందుకంటే rebase మీ commits ని target పైన replay చేస్తుంది. అందుకే rebase conflict లో గుడ్డిగా `--ours` వాడకు; content చూసి decide చెయ్.

VS Code లో conflict resolution (MERN devs కి practical)

VS Code conflicts ని highlight చేసి, ప్రతి conflict పైన buttons ఇస్తుంది: **Accept Current Change** (HEAD), **Accept Incoming Change**, **Accept Both Changes**, **Compare Changes**. Click చేసి resolve → save → `git add` → continue. GUI అయినా, లోపల అదే markers logic.

Key Points

- **Merge** = branchy history + merge commit (నిజం, safe). **Rebase** = linear history, commits rewritten (clean కానీ hashes మారతాయి).

- ⭐ **Rule: local private branches ని rebase, shared/public branches ని merge.** Pushed history ని ఎప్పుడూ rebase చేయకు.
- **Conflict** = ఇద్దరు అదే lines మార్చారు. Markers: `<<<<<<< HEAD` (నీది), `=======`, `>>>>>>> branch` (వచ్చేది).
- Resolve: markers తీసేసి correct version రాయి → `git add file` → `git commit` (merge) లేదా `git rebase --continue` (rebase).
- గందరగోళం అయితే **`git merge --abort` / `git rebase --abort`** – ముందటి safe స్థితికి.
- Rebase తర్వాత push కి **`--force-with-lease`** (history మారింది; safer than `--force`).

Interview దృష్టి

Q: Merge vs Rebase – ఏది ఎప్పుడు? A: Merge = actual history preserve చేస్తుంది, merge commit తో (safe, shared branches కి). Rebase = commits ని target పైన replay చేసి linear clean history ఇస్తుంది (కానీ hashes rewrite). Rule: నీ local feature branch ని main తో up-to-date చేయడానికి rebase (clean); feature ని main లోకి integrate చేయడానికి merge. **ఇతరులు pull చేసిన branch ని ఎప్పుడూ rebase చేయకు** – hashes మారి వారి history diverge అవుతుంది.

Q: Conflict ఎప్పుడు వస్తుంది, ఎలా resolve చేస్తావ్? A: రెండు branches లో అదే file యొక్క అదే (లేదా overlapping) lines మారితే – Git ఏది correct అని decide చేయలేదు. `<<<<<<< ======= >>>>>>>` markers పెడుతుంది. నేను file తెరిచి, రెండు versions చూసి, correct final version రాసి (ఒకటి/రెండూ/mix), markers తీసేసి, `git add` చేసి, merge అయితే `git commit`, rebase అయితే `git rebase --continue`. గందరగోళం అయితే `--abort` తో వెనక్కి.

Q: Rebase తర్వాత `git push` ఎందుకు reject అవుతుంది? A: Rebase commit hashes ని rewrite చేస్తుంది → local history remote history నుండి diverge అవుతుంది. Normal push "non-fast-forward" అని reject చేస్తుంది (remote work overwrite అవ్వకుండా). `--force-with-lease` తో push చేయాలి – ఇది remote నువ్వు చివరిసారి చూసిన స్థితిలోనే ఉంటే మాత్రమే overwrite చేస్తుంది (ఇంకెవరో push చేసి ఉంటే fail – safety). అందుకే rebase ని private branches కి మాత్రమే.

---

14. Remotes & Collaboration – push, pull, fetch, PR, fork

వివరణ

ఇప్పటివరకు అంతా local. కానీ team లో పని చేయాలంటే code ని **remote** (GitHub/GitLab) కి పంపాలి, ఇతరుల code తెచ్చుకోవాలి. **Remote** = నీ repo యొక్క cloud copy. `origin` = default remote పేరు. ఈ topic లో push/pull/fetch తేడా, tracking branches, PR workflow, మరియు open-source లో fork/upstream model.

Real-life Scenario

> **Remote = Google Drive లో shared team folder.**
>
> నీ laptop లో local copy (repo) ఉంది. Team అందరికీ కనిపించే cloud copy = **remote (origin)**.
>
> - **push** = నీ local మార్పులని cloud కి upload (team చూడగలరు).
> - **fetch** = cloud లో ఏం కొత్తగా వచ్చిందో *download చేసి చూడటం* (కానీ నీ working copy ముట్టకుండా).
> - **pull** = fetch + నీ copy లోకి merge (download + apply ఒకేసారి).
>
> **Fork** = ఇంకొకరి public folder యొక్క నీ సొంత copy తీసుకోవడం (వారి original ని ముట్టకుండా, నువ్వు మార్చుకోవడానికి). **Pull Request** = "నా మార్పులని నీ original folder లోకి తీసుకో?" అని request పంపడం – వారు review చేసి accept/reject.

Remote setup: `remote`, `origin`

```
git remote -v                   # ప్రస్తుత remotes చూడు
# origin  https://github.com/user/repo.git (fetch)
# origin  https://github.com/user/repo.git (push)

git remote add origin https://github.com/user/repo.git   # remote add
git remote add upstream https://github.com/original/repo.git  # 2nd remote (fork workflow)
git remote remove origin        # తీసేయి
git remote rename origin gh      # rename
git remote set-url origin git@github.com:user/repo.git   # HTTPS → SSH మార్చడం
```

> `origin` **అంటే ఏమిటి?** ఇది కేవలం remote కి **default పేరు** (magic కాదు). Clone చేసినప్పుడు Git automatic గా source ని `origin` అని పేరు పెడుతుంది. కావాలంటే వేరే పేరు పెట్టవచ్చు. Fork workflow లో రెండు remotes: `origin` (నీ fork), `upstream` (original repo).

`push` – local → remote

```
git push origin main            # main branch ని origin కి push
git push origin feature/login  # feature branch push
git push -u origin feature/login   # -u = upstream set (ఇక తర్వాత కేవలం 'git push' చాలు)
# Branch 'feature/login' set up to track 'origin/feature/login'.
git push                        # -u ఒకసారి చేశాక – tracking branch కి push
git push --force-with-lease     # rebase/amend తర్వాత (history మారితే)
git push origin --delete feature/old    # remote branch delete
git push origin --tags          # tags push (releases కి)
```

`fetch` vs `pull` – ⚠️ ముఖ్యమైన తేడా

```
# --- fetch: download మాత్రమే (safe — working copy ముట్టదు) ---
git fetch origin               # remote లో ఏం కొత్తగా వచ్చిందో download (origin/main
update)
git log origin/main            # download చేసినది చూడు (ఇంకా merge కాలేదు)
git diff main origin/main      # నా main vs remote main తేడా చూడు
git merge origin/main          # నచ్చితే merge

# --- pull: fetch + merge (ఒకేసారి) ---
git pull origin main           # = git fetch + git merge origin/main
git pull                       # tracking branch నుండి (upstream set అయితే)
git pull --rebase origin main  # merge బదులు rebase (linear history — చాలా teams
prefer)
git pull --ff-only             # fast-forward అయితేనే pull (merge commit వద్దు)
```

fetch vs pull:

అంశం	`git fetch`	`git pull`
ఏం చేస్తుంది	remote మార్పులు download మాత్రమే	fetch + merge (working copy లో apply)
working copy	ముట్టదు (safe)	మారుతుంది (merge/rebase)
conflicts	రావు (ఇంకా merge కాలేదు)	రావచ్చు
ఎప్పుడు	ముందు చూసి decide చేయాలంటే	direct update కావాలంటే

> **SSE habit:** `git pull` blindly చేయడం కంటే, `git fetch` చేసి `git log origin/main` / `git diff` తో ఏం వచ్చిందో చూసి, తర్వాత merge/rebase – ఇది safer (surprise conflicts తగ్గుతాయి). `git pull --rebase` = pull చేసేటప్పుడు merge commit కాకుండా నీ commits ని పైన పెడుతుంది (clean history).

Tracking branches

Tracking branch = local branch ↔ remote branch link. Set అయితే `git push` / `git pull` కి branch పేరు type చేయనవసరం లేదు.

```
git push -u origin feature/login    # -u = tracking set చేస్తుంది
git branch -vv                       # ఏ local branch ఏ remote ని track చేస్తోందో
# * feature/login  a3f5c9e [origin/feature/login] feat: add login
#   main           b2d4a1f [origin/main] init
git branch --set-upstream-to=origin/main main   # manually tracking set
```

⭐ PR (Pull Request) workflow – teams ఇలా పని చేస్తాయి

Production లో direct `main` కి push చేయరు. Feature branch → PR → review → merge. Standard flow:

```
# 1. latest main నుండి కొత్త branch
git switch main
git pull origin main
git switch -c feature/add-cart

# 2. పని + commits
git add . && git commit -m "feat: add shopping cart"

# 3. push (upstream set)
git push -u origin feature/add-cart

# 4. GitHub లో "Compare & Pull Request" → PR create
#    - reviewers assign, description రాయి
#    - CI/tests automatic run అవుతాయి
#    - teammates review, comments, approve

# 5. approve అయ్యాక GitHub లో "Merge" (లేదా "Squash and merge")

# 6. local cleanup
git switch main
git pull origin main           # merged మార్పులు తెచ్చుకో
git branch -d feature/add-cart  # local branch delete
git push origin --delete feature/add-cart  # remote branch delete
```

> **PR ఎందుకు?** (1) **Code review** – bugs early పట్టుకోవడం, knowledge sharing. (2) **CI gate** – tests/lint pass అయితేనే merge. (3) **Discussion** – design decisions record. (4) **main protection** – broken code direct main కి వెళ్ళదు. SSE interview లో "మీ team git workflow ఏమిటి?" అంటే ఇదే చెప్పాలి.

Fork & Upstream – open-source contribution

Open-source repo కి నీకు write access ఉండదు. అందుకే **fork** (నీ సొంత copy) చేసి, అందులో పని చేసి, PR పంపుతావు.

```
original repo (upstream)  ──fork──▶  నీ fork (origin, GitHub లో నీ account)
                                          │
                                      git clone
                                          ▼
                                     నీ local copy
```

```
# 1. GitHub లో "Fork" button → నీ account లో copy

# 2. నీ fork ని clone
git clone https://github.com/YOUR-USERNAME/repo.git
cd repo

# 3. original ని 'upstream' remote గా add (updates తెచ్చుకోవడానికి)
git remote add upstream https://github.com/ORIGINAL-OWNER/repo.git
git remote -v
# origin    https://github.com/YOUR-USERNAME/repo.git  (నీ fork)
# upstream  https://github.com/ORIGINAL-OWNER/repo.git (original)
```

```
# 4. original నుండి latest తెచ్చుకో (అది మారుతూ ఉంటుంది)
git fetch upstream
git switch main
git merge upstream/main        # లేదా rebase
git push origin main           # నీ fork ని కూడా update

# 5. feature branch → పని → push నీ fork కి → GitHub నుండి original కి PR
git switch -c fix/typo
# ... commits ...
git push -u origin fix/typo
#    GitHub లో "Compare across forks" → original repo కి PR
```

origin vs upstream:

Remote	ఏమిటి	పని
origin	నీ fork (నీ GitHub account)	నువ్వు push చేసేది
upstream	original repo	latest updates fetch చేసేది (push చేయవు)

Key Points

- **Remote** = repo యొక్క cloud copy; `origin` = default remote పేరు (clone చేసినప్పుడు auto).
- `push` = local → remote; `fetch` = remote download మాత్రమే (safe); `pull` = fetch + merge (working copy మారుతుంది).
- `git push -u` ఒకసారి → tracking branch set → తర్వాత కేవలం `git push` / `git pull`.
- **PR workflow:** branch → push → PR → review + CI → merge → cleanup. Direct main కి push చేయరు.
- **Fork model:** `origin` (నీ fork) + `upstream` (original). Open-source contribution కి standard.
- `git fetch` + review + merge = blind `git pull` కంటే safer.

Interview దృష్టి

Q: `git fetch` మరియు `git pull` తేడా? A: `fetch` = remote మార్పులని download చేసి `origin/main` లాంటి remote-tracking branches ని update చేస్తుంది, కానీ నీ working branch ని **ముట్టదు** (safe – ముందు చూసి decide చేయవచ్చు). `pull` = `fetch` + `merge` (లేదా `--rebase`) – download చేసి వెంటనే నీ current branch లోకి integrate చేస్తుంది (conflicts రావచ్చు). `pull` = `fetch` + `merge` రెండూ ఒకేసారి.

Q: PR workflow ఎందుకు, direct main కి push ఎందుకు కాదు? A: PR = code review + CI gate + discussion + main branch protection. Direct push చేస్తే – review లేదు (bugs slip), broken code production కి వెళ్తుంది, knowledge silo అవుతుంది. PR తో teammates review చేస్తారు, tests automatic run అవుతాయి, approve అయితేనే merge. Quality + collaboration + safety. చాలా teams `main` ని "protected" చేస్తాయి (direct push disable).

Q: `origin` మరియు `upstream` తేడా (fork workflow)? A: Open-source లో original repo కి write access ఉండదు. `origin` = నీ **fork** (నీ GitHub account లో copy – నువ్వు push చేసేది). `upstream` =

original repo (latest changes fetch చేసుకోవడానికి; push చేయవు). నువ్వు origin కి push చేసి, original కి PR పంపుతావు; upstream నుండి regularly fetch చేసి నీ fork ని sync లో ఉంచుతావు.

---

15. Undoing Mistakes – reset, revert, restore, stash, amend, reflog

వివరణ

ఇది SSE ని junior నుండి వేరు చేసే topic. అందరూ commit చేస్తారు; కానీ "తప్పు commit చేశాను", "main కి తప్పు push అయ్యింది", "కోడ్ accidentally delete చేశాను", "commit message తప్పు" – వీటిని **confident గా undo** చేయడం seniority signal. Git లో దాదాపు ప్రతిదీ recoverable – భయపడాల్సిన అవసరం లేదు (reflog ఉంది!). ఒక్కొక్క scenario కి సరైన tool చూద్దాం.

Real-life Scenario

> **Git undo tools = వేర్వేరు "వెనక్కి తీసుకునే" విధానాలు.**
>
> - `restore` = draft లో రాసిన paragraph ని చెరిపేసి, చివరి save కి తిరిగి రావడం (commit కాని మార్పులు).
> - `reset` = time machine – "ఈ commit కి తిరిగి వెళ్ళు". `--soft` = మార్పులు draft లో ఉంచు; `--hard` = మార్పులు కూడా చెరిపేయి (⚠️).
> - `revert` = తప్పు entry ని చెరపకుండా, "ఆ entry ని రద్దు చేస్తున్నా" అని కొత్త entry రాయడం (public history కి safe).
> - `stash` = సగం రాసిన పని ని ఒక drawer లో దాచి, table clean చేసి, తర్వాత తీసుకోవడం.
> - `amend` = ఇప్పుడే రాసిన చివరి entry ని సరిదిద్దడం (typo fix).
> - `reflog` = "నేను గత రెండు రోజుల్లో ఎక్కడెక్కడికి వెళ్ళానో" CCTV record – పోయిన commits కూడా ఇక్కడ కనిపిస్తాయి (life-saver!).

`git restore` – working directory మార్పులు undo

```
git restore app.js              # app.js లో commit కాని మార్పులు రద్దు (last commit కి)
git restore .                   # అన్ని unstaged మార్పులు రద్దు (⚠️ పోతాయి – undo లేదు)
git restore --staged app.js     # staged → unstaged (git add ని undo, మార్పులు ఉంటాయి)
git restore --source=HEAD~2 app.js   # 2 commits వెనుక version తెచ్చు
```

> `restore` **= ఆధునికం.** పాతగా `git checkout -- file` (files) మరియు `git reset HEAD file` (unstage) వాడేవారు – overloaded, confusing. Git 2.23 లో `restore` వచ్చింది: file content restore + staging control ఒకే command లో, clear గా. కానీ `--staged` లేని `git restore` **destructive** (commit కాని మార్పులు పోతాయి).

`git commit --amend` – చివరి commit సరిదిద్దడం

```
git commit --amend -m "feat: correct message"   # చివరి commit message మార్చు
git add forgotten.js
git commit --amend --no-edit   # చివరి commit కి forgotten file కలుపు (message అలాగే)
```

> ⚠️ **amend = కొత్త commit (hash మారుతుంది).** ఇది చివరి commit ని replace చేస్తుంది. **Push చేయని** commit కి safe. Push అయ్యాక amend చేస్తే – `--force-with-lease` అవసరం, shared branch అయితే ప్రమాదం (team history diverge).

`git reset` – commits ని వెనక్కి తీయడం (soft/mixed/hard)

`reset` branch pointer ని ఒక commit వెనక్కి కదిలిస్తుంది. **మూడు modes** – ప్రధాన తేడా: staging, working directory ని ఏం చేస్తుంది.

```
git reset --soft HEAD~1        # 1 commit undo; మార్పులు STAGED గా ఉంటాయి
git reset --mixed HEAD~1       # 1 commit undo; మార్పులు WORKING DIR లో (unstaged)
[default]
git reset HEAD~1               # = --mixed (default)
git reset --hard HEAD~1        # ⚠️ 1 commit undo; మార్పులు కూడా చెరిగిపోతాయి (గోన్)
git reset --hard origin/main   # local ని remote కి exactly match (local commits
పోతాయి)
git reset a3f5c9e              # ఒక specific commit కి reset (mixed)
```

మూడు modes పోలిక (interview must-know):

Mode	Commit undo?	Staging	Working Directory	ఎప్పుడు
`--soft`	✅	మార్పులు **staged** గా ఉంచుతుంది	ముట్టదు	commits ని re-organize/combine
`--mixed` (default)	✅	unstaged (add undo)	మార్పులు **ఉంచుతుంది**	commit + add ని undo, మార్పులు keep
`--hard`	✅	చెరిపేస్తుంది	**చెరిపేస్తుంది** ⚠️	పూర్తిగా వదిలేయాలంటే (danger!)

```
         Repository ← Staging ← Working Directory
--soft:    reset       keep         keep         (మార్పులు staged గా మిగులుతాయి)
--mixed:   reset       reset        keep         (మార్పులు unstaged గా మిగులుతాయి)
--hard:    reset       reset        reset   ⚠️   (అంతా పోతుంది)
```

Common use – చివరి commit ని "un-commit" చేసి తిరిగి రాయడం:

```
git reset --soft HEAD~1        # commit undo, మార్పులు staged గా – మళ్ళీ commit చేయవచ్చు
# (ఉదా: 3 చిన్న commits ని ఒక్కటిగా చేయాలంటే: git reset --soft HEAD~3 → git commit)
```

> ⚠️ `--hard` = **అత్యంత ప్రమాదకరం.** commit కాని మార్పులు permanently పోతాయి. Push అయిన commits ని reset చేసి force-push చేస్తే team history చెడిపోతుంది. **Rule:** `reset --hard` ముందు `git status` చూడు, ముఖ్యమైనవి ఉంటే ముందు `git stash`. Committed మార్పులైతే `reflog` నుండి recover చేయవచ్చు, కానీ uncommitted అయితే గోన్.

`git revert` – public history కి safe undo

`reset` history ని rewrite చేస్తుంది (shared branch కి ప్రమాదం). `revert` బదులుగా – తప్పు commit ని **రద్దు చేసే కొత్త commit** create చేస్తుంది (history అలాగే ఉంటుంది).

```
git revert a3f5c9e          # a3f5c9e మార్పులని రద్దు చేసే కొత్త commit
git revert HEAD             # చివరి commit ని రద్దు
git revert HEAD~2..HEAD     # range revert
git revert -n a3f5c9e       # -n = revert but commit చేయకు (multiple కలిపి commit
చేయడానికి)
```

reset vs revert (⭐ interview classic):

అంశం	`reset`	`revert`
ఏం చేస్తుంది	branch pointer వెనక్కి (commits తొలగిస్తుంది)	undo చేసే **కొత్త commit** create
history	**rewrite** అవుతుంది	preserve (కొత్త commit కలుస్తుంది)
shared branch కి	⚠️ ప్రమాదం (force-push అవసరం)	✅ safe (force అవసరం లేదు)
ఎప్పుడు	local, unpushed commits	**already-pushed / main** commits

> **Rule:** Push చేయని local commits ని సరిదిద్దాలంటే `reset`. Already-pushed (main/shared) commit ని undo చేయాలంటే `revert` (history rewrite చేయకుండా, force-push అవసరం లేకుండా). "Production లో ఒక commit bug తెచ్చింది – undo చెయ్" = `git revert`.

`git stash` – పని ని పక్కన పెట్టడం

సగం పని చేస్తున్నావు, అంతలో "urgent bug fix చెయ్" అని వచ్చింది. Commit చేయడానికి పని పూర్తి కాలేదు. `stash` = మార్పులని temporary గా దాచి, clean working dir ఇస్తుంది.

```
git stash                   # ప్రస్తుత మార్పులు (tracked) ని దాచు → working dir clean
git stash push -m "half cart"  # message తో
git stash -u                # -u = untracked files కూడా stash
git stash list              # దాచిన stashes
# stash@{0}: On feature: half cart
# stash@{1}: WIP on main: ...
git stash pop               # చివరి stash తిరిగి తెచ్చు + list నుండి తీసేయి
git stash apply             # తిరిగి తెచ్చు కానీ list లో ఉంచు (multiple చోట్ల apply
చేయడానికి)
git stash apply stash@{1}   # specific stash
git stash drop stash@{0}    # ఒక stash తీసేయి
git stash clear             # అన్ని stashes తీసేయి
```

Stash use case:

```
# feature మీద పని చేస్తున్నా, urgent hotfix రావాలి:
git stash                       # feature పని దాచు
git switch main
git switch -c hotfix/crash      # hotfix branch
# ... fix + commit + push ...
git switch feature/xyz          # తిరిగి feature కి
git stash pop                   # దాచిన పని తిరిగి తెచ్చు
```

`git cherry-pick` – ఒక specific commit ని తీసుకోవడం

మరో branch లోని ఒక్క commit ని (మొత్తం branch కాదు) నీ current branch కి తీసుకోవాలంటే:

```
git cherry-pick a3f5c9e         # ఆ commit ని current branch కి apply
git cherry-pick a3f5c9e b2d4a1f     # multiple commits
git cherry-pick main~2          # main యొక్క 2-వెనుక commit
git cherry-pick -x a3f5c9e      # -x = original commit hash ని message లో note చేయి
```

> **Use case:** hotfix ని `main` కి apply చేశావు, అదే fix `release-2.0` branch కి కూడా కావాలి – మొత్తం merge అవసరం లేదు, ఆ ఒక్క commit ని `cherry-pick` చెయ్. లేదా తప్పు branch లో commit చేశావు – సరైన branch కి cherry-pick చేసి, తప్పు branch నుండి reset.

⭐ `git reflog` – life-saver (పోయినది కూడా తిరిగి తెస్తుంది)

reflog = HEAD ఎక్కడెక్కడికి కదిలిందో ప్రతి కదలిక record (commits, resets, checkouts, rebases). Reset --hard చేసి commits పోగొట్టుకున్నా, reflog లో వాటి hash ఉంటుంది → recover చేయవచ్చు!

```
git reflog                      # HEAD యొక్క అన్ని కదలికలు
# a3f5c9e HEAD@{0}: reset: moving to HEAD~1
# b2d4a1f HEAD@{1}: commit: feat: add feature   ← reset తో పోయిన commit!
# c1e8f2a HEAD@{2}: commit: init

# పోయిన commit ని recover:
git reset --hard b2d4a1f        # ఆ commit కి తిరిగి వెళ్ళు (recovered!)
# లేదా కొత్త branch లో:
git branch recovered b2d4a1f
```

> **reflog ఎందుకు life-saver?** `git reset --hard`, botched rebase, deleted branch – వీటిలో commits "పోయినట్టు" అనిపిస్తాయి కానీ Git వాటిని వెంటనే delete చేయదు (~90 రోజులు reflog + garbage collection వరకు ఉంచుతుంది). reflog లో hash కనిపెట్టి recover చేయవచ్చు. **అందుకే Git లో దాదాపు ఏదీ నిజంగా పోదు** – panic అవ్వకు, reflog చూడు. (Note: reflog local మాత్రమే – clone చేసినప్పుడు రాదు.)

⭐ Scenario → tool cheat sheet

సమస్య	పరిష్కారం
Commit కాని file మార్పులు వద్దు	`git restore <file>`
`git add` ని undo (unstage)	`git restore --staged <file>`
చివరి commit message తప్పు	`git commit --amend -m "..."`
చివరి commit కి file మర్చిపోయా	`git add f && git commit --amend --no-edit`
చివరి commit ని un-commit (మార్పులు keep)	`git reset --soft HEAD~1`
చివరి కొన్ని commits పూర్తిగా వదిలేయాలి	`git reset --hard HEAD~N` (⚠️)
Pushed commit ని undo (safe)	`git revert <hash>`
సగం పని పక్కన పెట్టాలి	`git stash` → `git stash pop`
మరో branch commit ఇక్కడ కావాలి	`git cherry-pick <hash>`
Reset/delete తో పోయిన commit	`git reflog` → `git reset --hard <hash>`

Key Points

- **`restore`** = working dir/staging undo; **`amend`** = చివరి commit fix (hash మారుతుంది).
- **`reset`** modes: `--soft` (commit undo, staged keep), `--mixed` (unstaged keep – default), `--hard` (అంతా చెరుపు ⚠️).
- **`reset` vs `revert`:** reset = history rewrite (local only); **revert = safe undo via కొత్త commit (pushed/main కి).**
- **`stash`** = పని పక్కన పెట్టడం; **`cherry-pick`** = ఒక specific commit తీసుకోవడం.
- ⭐ **`reflog`** = HEAD కదలికల record – reset/rebase తో పోయిన commits recover చేయవచ్చు. Git లో దాదాపు ఏదీ పోదు.

Interview దృష్టి

Q: `git reset` మూడు modes తేడా? A: మూడూ branch pointer ని వెనక్కి కదిలిస్తాయి, కానీ staging + working dir ని వేర్వేరుగా చూస్తాయి. `--soft`: commit undo, మార్పులు **staged** గా ఉంటాయి (re-commit కి ready). `--mixed` (default): commit + add undo, మార్పులు **working dir** లో unstaged. `--hard`: commit + మార్పులు అన్నీ **చెరిపేస్తుంది** (uncommitted work గోన్ – ప్రమాదకరం).

Q: `reset` vs `revert` – ఏది ఎప్పుడు? A: `reset` history ని rewrite చేస్తుంది (commits తొలగిస్తుంది) – **local, unpushed** commits కి మంచిది; shared branch కి force-push అవసరం (ప్రమాదం). `revert` original commit ని ముట్టకుండా, దాన్ని రద్దు చేసే **కొత్త commit** create చేస్తుంది – **already-pushed/main** commits కి safe (history preserve, force-push అవసరం లేదు). "Production లో ఒక commit undo" = revert.

Q: `git reset --hard` చేసి commits పోగొట్టుకున్నావ్ – recover చేయగలవా? A: అవును (committed అయితే). `git reflog` లో HEAD యొక్క ప్రతి కదలిక record ఉంటుంది – పోయిన commit

hash కనిపిస్తుంది. `git reset --hard <hash>` లేదా `git branch recovered <hash>` తో తిరిగి తెచ్చుకోవచ్చు. Git commits ని వెంటనే delete చేయదు (~90 రోజులు). కానీ **uncommitted** మార్పులు (`add` కూడా చేయనివి) reflog లో ఉండవు – అవి recover కావు. అందుకే `--hard` ముందు `stash`.

---

16. Git Workflows – feature-branch, gitflow, trunk-based + best practices

వివరణ

Commands తెలిస్తే సరిపోదు – team ఏ **workflow** (branching strategy) follow చేస్తుందో తెలియాలి. Workflow = "branches ని ఎలా organize చేయాలి, ఎప్పుడు merge చేయాలి, release ఎలా చేయాలి" అనే team agreement. SSE interview లో "మీ team git workflow ఏమిటి?" అని తప్పకుండా అడుగుతారు. మూడు main strategies చూద్దాం.

Real-life Scenario

> **Git workflow = ఒక restaurant kitchen లో order handling system.**
>
> - **Feature-branch** = ప్రతి order (feature) కి separate station, ready అయ్యాక main counter కి. Simple, చిన్న teams కి.
> - **Gitflow** = పెద్ద hotel – separate prep kitchen (`develop`), plating station (`release`), emergency counter (`hotfix`), final serving (`main`). Structured, formal releases కి. కానీ heavy.
> - **Trunk-based** = fast-food express – అందరూ ఒకే counter (`trunk`) దగ్గర చిన్న చిన్న orders వేగంగా పంపుతారు, feature flags తో "ఇంకా ready కాని" items దాచుతారు. Continuous delivery కి.
>
> ఏ system best? Restaurant size + release frequency ని బట్టి. Startup MERN app = feature-branch/trunk-based; enterprise scheduled releases = gitflow.

1. Feature-Branch Workflow (అత్యంత common, startups/MERN)

ఆలోచన: `main` ఎప్పుడూ deployable. ప్రతి feature/fix కి `main` నుండి ఒక branch, పని అయ్యాక PR → review → merge.

```
main:      ──●────●────●────●───▶   (ఎప్పుడూ stable, deployable)
             \        /  \    /
feature-A:    ●──●──●     |           (PR → merge)
feature-B:           ●──●──●           (PR → merge)
```

```
git switch main && git pull
git switch -c feature/add-search    # main నుండి branch
# ... పని + commits ...
```

```
git push -u origin feature/add-search
# GitHub PR → review → merge → branch delete
```

✅ లాభాలు	❌ నష్టాలు
Simple, అర్థం చేసుకోవడం సులభం	పెద్ద branches → merge conflicts పెరుగుతాయి
main ఎప్పుడూ stable	long-lived branches drift అవుతాయి
PR review + CI natural	formal release process లేదు

> **MERN teams కి default.** చిన్న/మధ్య teams, continuous deployment. GitHub Flow అనేది దీని simplest form (main + short feature branches + PR).

2. Gitflow (structured, scheduled releases)

ఆలోచన: బహుళ long-lived branches, ప్రతిదానికి specific role. Formal versioned releases (v1.0, v2.0) ఉన్న products కి.

Branch	పని	lifespan
`main`	production code (ప్రతి merge = release, tagged)	permanent
`develop`	integration branch (అన్ని features ఇక్కడ కలుస్తాయి)	permanent
`feature/*`	కొత్త features (develop నుండి, develop కి)	temporary
`release/*`	release తయారీ (bug fixes, version bump)	temporary
`hotfix/*`	production urgent fix (main నుండి, main+develop కి)	temporary

```
main:     ●────────────────●────────●──▶   (releases: v1.0, v1.1)
           \              / \      /
release:    \       ●──●─┘   |    /
             \     /         |   /
develop:  ●───●───●───●───●───●──▶
           \  /    \   /
feature:    ●       ●─●
```

```
git switch develop
git switch -c feature/payments     # feature develop నుండి
# ... పని ...  → develop లోకి merge
git switch -c release/1.2 develop  # release తయారీ
# ... final fixes, version bump ...
git switch main && git merge release/1.2 && git tag v1.2
git switch develop && git merge release/1.2   # develop కి కూడా
```

✅ లాభాలు	❌ నష్టాలు
structured, formal releases	complex (చాలా branches)
parallel release + hotfix support	continuous delivery కి overkill
versioned products కి perfect	slow, merge overhead ఎక్కువ

> **ఎప్పుడు:** scheduled releases, multiple versions maintain చేసే products (mobile apps, enterprise software, on-premise). Modern web/MERN కి చాలామంది దీన్ని too heavy అంటారు.

3. Trunk-Based Development (CI/CD, large scale)

ఆలోచన: అందరూ ఒకే branch (`trunk` / `main`) కి చాలా చిన్న changes రోజూ commit చేస్తారు (branches చాలా short-lived, గంటలు/రోజు). "ఇంకా ready కాని" features ని **feature flags** తో దాచుతారు.

```
main:  ──●─●─●─●─●─●─●─●─●──▸  (అందరూ ఇక్కడికి రోజూ, చిన్న commits)
          \/   \/   \/
   (గంటల short branches, వెంటనే merge)
```

```
git switch -c quick-fix           # చాలా short branch
# ... చిన్న change + commit ...
git switch main && git merge quick-fix && git push   # అదే రోజు merge
# ready కాని feature → code లో ఉంటుంది కానీ feature flag off:
# if (featureFlags.newCheckout) { ... }
```

✅ లాభాలు	❌ నష్టాలు
merge conflicts చాలా తక్కువ (చిన్న diffs)	strong CI/CD + tests తప్పనిసరి
continuous integration/delivery	feature flags discipline కావాలి
Google, Facebook scale కి	incomplete code main లో (flags తో దాచాలి)

> **ఎప్పుడు:** mature teams, strong automated testing, continuous deployment. Google/Meta ఇదే వాడతాయి. Feature flags + CI లేకపోతే risky.

Workflows comparison

అంశం	Feature-Branch	Gitflow	Trunk-Based
complexity	తక్కువ	ఎక్కువ	తక్కువ (కానీ discipline)
branch lifespan	రోజులు	వారాలు	గంటలు
release style	continuous	scheduled/versioned	continuous

అంశం	Feature-Branch	Gitflow	Trunk-Based
best for	most startups/MERN	enterprise, versioned	CI/CD mature teams
conflicts	medium	ఎక్కువ	తక్కువ

⭐ Git best practices (SSE గా follow చేయాలి)

Commits:

- **Atomic commits** – ఒక commit = ఒక logical change. "fix login + add footer + update deps" ని ఒకే commit చేయకు.
- **Meaningful messages** – `feat: add JWT refresh token rotation`, `"changes"` కాదు. Present tense, imperative.
- **తరచుగా commit** చెయ్, కానీ **push చేసేది clean** గా (rebase -i తో WIP commits cleanup).

Branches:

- **Short-lived branches** – పెద్ద/పాత branches = merge hell. త్వరగా merge చెయ్.
- **Naming convention** – `feature/`, `fix/`, `hotfix/`, `chore/` prefix. `feature/JIRA-123-add-cart`.
- **main ని protect** చెయ్ – direct push disable, PR + review + CI mandatory.

Collaboration:

- **Pull before push** – `git pull --rebase` regularly (conflicts early).
- **PRs చిన్నగా** ఉంచు – 200-400 lines review చేయడం సులభం; 2000 lines review దాటవేస్తారు.
- `.gitignore` **సరిగ్గా** – node_modules, .env, dist ఎప్పుడూ ignore.
- **Secrets ఎప్పుడూ commit చేయకు** – `.env`, keys. Accidentally అయితే rotate చేయి.

Safety:

- **Force-push ఎప్పుడూ** `--force-with-lease` – shared branches కి force అస్సలు వద్దు.
- `reset --hard` **ముందు** `stash` – uncommitted work backup.
- **Rebase కేవలం local/private** branches కి.

Key Points

- **Feature-Branch** = main stable + short feature branches + PR. **Most startups/MERN కి default.**
- **Gitflow** = main/develop/feature/release/hotfix – structured, scheduled/versioned releases కి (కానీ heavy).
- **Trunk-Based** = అందరూ main కి చిన్న commits + feature flags – CI/CD mature teams కి (Google/Meta).
- Best practices: **atomic commits, meaningful messages, short-lived branches, small PRs, protected main, no secrets, `--force-with-lease`.**
- Workflow ఎంపిక = team size + release frequency + CI/CD maturity.

Interview దృష్టి

Q: మీ team ఏ git workflow వాడుతుంది, ఎందుకు? A: (నిజాయితీగా చెప్పు) "మేము feature-branch workflow వాడతాము – main ఎప్పుడూ deployable, ప్రతి feature/fix కి main నుండి short-lived branch, PR తో code review + CI checks, approve అయ్యాక squash-merge. మా team చిన్నది + continuous deployment కాబట్టి gitflow overkill, ఇది simple + safe." – reasoning ముఖ్యం, workflow name కాదు.

Q: Gitflow ఎప్పుడు వాడతావ్, ఎప్పుడు వద్దు? A: Gitflow = scheduled/versioned releases, multiple versions maintain చేసే products (mobile apps, enterprise, on-premise software) కి మంచిది – release/hotfix branches parallel work support చేస్తాయి. కానీ modern web/SaaS (రోజుకి multiple deploys) కి too heavy – చాలా branches, merge overhead. Continuous delivery కి feature-branch లేదా trunk-based better.

Q: మంచి commit ఎలా ఉండాలి? A: **Atomic** (ఒక logical change), **meaningful message** (`type: what and why`, imperative), independently understandable/revertable. చెడు = 20 files unrelated changes + `"stuff"` message. మంచి = login fix ఒక commit, footer style మరో commit, ప్రతిదానికి clear message. ఇది code review, debugging (`git bisect`), revert అన్నిటికీ సహాయం.

Part 4 – Reference (Interview రోజు)

> Interview ముందు రోజు రాత్రి ఈ Part చదువు – rapid revision. అన్ని topics నుండి అత్యంత అడిగే Q&A, రోజూ వాడే commands cheat-sheet, మరియు అందరూ చేసే mistakes (వాటిని నువ్వు తప్పించుకో). ఇది memorize కాదు – పైన నేర్చుకున్నది గుర్తు తెచ్చుకునే index.

---

17. Interview Q&A + Command Cheat-sheet + Common Mistakes

⭐ Rapid-fire Interview Q&A

Linux:

Q: Kernel అంటే ఏమిటి? A: OS యొక్క core – hardware (CPU, memory, disk, network) ని software తో మాట్లాడించే layer. Process scheduling, memory management, file systems, device drivers, system calls అన్నీ kernel పని. Linux = kernel; Ubuntu = kernel + tools = distro.

Q: `chmod 755` అంటే ఏమిటి? A: rwx (owner=7), r-x (group=5), r-x (others=5). Owner అన్నీ చేయవచ్చు, మిగతా వాళ్ళు read+execute కానీ write కాదు. Scripts, folders, executables కి typical. `644` = files (owner rw, others r), `600` = secrets (owner only).

Q: Process ని ఎలా kill చేస్తావ్, `-9` ఎప్పుడు? A: `kill <PID>` = SIGTERM (graceful, cleanup కి time). పనిచేయకపోతేనే `kill -9` (SIGKILL, force, cleanup లేదు). ఎప్పుడూ SIGTERM మొదట – data corruption avoid చేయడానికి. Port block అయితే `lsof -i :3000` → kill.

Q: `>` మరియు `>>` తేడా? A: `>` = overwrite (file content తుడిచి రాస్తుంది), `>>` = append (చివర కలుపుతుంది). `2>` = stderr redirect, `2>&1` = stderr ని stdout ఉన్న చోటికి.

Q: `grep` vs `find`? A: `grep` = files లోపల content (text pattern) వెతుకుతుంది. `find` = files ని పేరు/size/date/type ప్రకారం filesystem లో వెతుకుతుంది. కలిపి: `find . -name "*.js" | xargs grep "TODO"`.

Q: `ssh` ఎలా secure? A: Key pair (asymmetric crypto). Private key client, public key server. Server challenge పంపుతుంది, client private key తో sign, server public key తో verify. Private key network మీద వెళ్ళదు. Session encrypted.

Q: `export VAR` ఎందుకు? A: `export` లేకుండా variable current shell కి మాత్రమే; child processes (node, python) కి కనిపించదు. `export` = child processes కి inherit. `process.env.VAR` చదవాలంటే export తప్పనిసరి.

Q: `scp` vs `rsync`? A: `scp` = ప్రతిసారి అన్నీ copy. `rsync` = మారిన/కొత్త files మాత్రమే (delta) – repeated deploys కి fast, `--exclude` / `--delete` /resume support. Deploy కి rsync better.

Git:

Q: Git vs GitHub? A: Git = version control *tool* (local, offline). GitHub = git repos ని host చేసే cloud *platform* (PRs, review, CI). కెమెరా vs Google Photos.

Q: 3 areas ఏమిటి? A: Working Directory (actual files) → `git add` → Staging Area (commit కి ready) → `git commit` → Repository (permanent history).

Q: `git fetch` vs `git pull`? A: `fetch` = remote మార్పులు download మాత్రమే (working copy safe). `pull` = fetch + merge (working copy లో apply, conflicts రావచ్చు). `pull` = `fetch` + `merge`.

Q: Merge vs Rebase? A: Merge = merge commit, branchy history (safe, shared). Rebase = commits ని target పైన replay, linear history (hashes rewrite). Rule: local branches rebase, shared branches merge. Pushed history ఎప్పుడూ rebase చేయకు.

Q: `reset` vs `revert`? A: `reset` = history rewrite (commits తొలగించు) – local/unpushed కి. `revert` = undo చేసే కొత్త commit (history preserve) – pushed/main కి safe.

Q: `reset` మూడు modes? A: `--soft` (commit undo, staged keep), `--mixed` (unstaged keep, default), `--hard` (అంతా చెరుపు ⚠️).

Q: Conflict ఎలా resolve? A: `<<<<<<< HEAD` (నీది) / `=======` / `>>>>>>> branch` (వచ్చేది) markers. Correct version రాసి markers తీసేసి → `git add` → `git commit` (merge) / `git rebase --continue` (rebase).

Q: పోయిన commit recover? A: `git reflog` → HEAD కదలికల record లో పోయిన commit hash → `git reset --hard <hash>`. Committed అయితే ~90 రోజులు recoverable.

Q: Fast-forward merge? A: Target branch మారకపోతే – merge commit లేకుండా pointer ముందుకు జరుపుతుంది (linear). Diverge అయితే 3-way merge (merge commit, 2 parents).

📋 Command Cheat-Sheet

Linux navigation & files:

Command	పని
`pwd`	ఎక్కడ ఉన్నాను
`ls -la`	files (hidden సహా), long format
`cd ~ / .. / -`	home / parent / previous
`mkdir -p a/b/c`	nested folders
`touch f`	ఖాళీ file / timestamp
`cp -r src dst`	copy (folder recursive)
`mv a b`	move / rename

Command	పని
`rm -rf dir`	⚠️ delete (permanent)
`ln -s target link`	symlink
`cat / less / head / tail -f`	view / page / top / live

Permissions & processes:

Command	పని
`chmod 755 f` / `chmod +x f`	permissions / executable
`chown user:grp f`	ownership
`ps aux | grep node`	processes వెతుకు
`top` / `htop`	live monitor
`kill <PID>` / `kill -9`	graceful / force
`lsof -i :3000`	port ఎవరు వాడుతున్నారు
`systemctl start/restart/status x`	service manage
`journalctl -u x -f`	service logs live

Text tools:

Command	పని
`grep -rn "text" .`	recursive search + line#
`find . -name "*.js"`	files వెతుకు
`sed -i 's/a/b/g' f`	find & replace (in-place)
`awk '{print $1}'`	column extract
`sort | uniq -c | sort -rn`	frequency count
`cmd | xargs rm`	output → args
`cmd > f 2>&1`	output + errors → file

Networking & env:

Command	పని
`curl -X POST -H .. -d ..`	API request

Command	పని
`ssh user@host`	remote login
`scp / rsync -avz`	file transfer / smart sync
`ss -tulpn`	open ports
`export VAR=x`	env variable
`echo $PATH` / `which node`	path / command location
`source ~/.bashrc`	reload config

Git daily:

Command	పని
`git clone / init`	repo తెచ్చు / సృష్టించు
`git status / log --oneline`	state / history
`git add . / commit -m`	stage / record
`git diff / diff --staged`	unstaged / staged మార్పులు
`git switch -c branch`	branch create + switch
`git merge / rebase branch`	integrate
`git push -u origin br` / `pull`	upload / download+merge
`git stash / pop`	పని పక్కన / తిరిగి

Git undo:

Command	పని
`git restore f`	file మార్పులు undo
`git restore --staged f`	unstage
`git commit --amend`	చివరి commit fix
`git reset --soft/--mixed/--hard HEAD~1`	commit undo (3 modes)
`git revert <hash>`	safe undo (కొత్త commit)
`git reflog`	పోయిన commits recover
`git cherry-pick <hash>`	ఒక commit తీసుకో

⚠️ Common Mistakes (అందరూ చేసేవి – నువ్వు తప్పించుకో)

Linux:

1. `rm -rf` **గుడ్డిగా** – path double-check చేయకుండా. Variable ఖాళీ అయితే `rm -rf $DIR/` = `rm -rf /`. ఎప్పుడూ `pwd` + path verify.
2. **root గా అన్నీ చేయడం** – `sudo` ప్రతిదానికీ. Normal user గా ఉండి అవసరమైనప్పుడే sudo.
3. `npm install` **ని sudo తో** – node_modules root-owned అవుతుంది, తర్వాత permission errors. Project లో ఎప్పుడూ sudo లేకుండా.
4. **Variables ని quote చేయకపోవడం** – `rm $file` (space ఉంటే bug). ఎప్పుడూ `"$file"`.
5. `kill -9` **మొదట** – graceful SIGTERM try చేయకుండా. Data corruption risk.
6. **పెద్ద file కి** `cat` – terminal freeze. `less` / `tail` వాడు.

Git:

1. `.env` **/secrets commit చేయడం** – `.gitignore` లో పెట్టు. Accidentally అయితే rotate + history clean.
2. `node_modules` **commit** – repo bloat. `.gitignore` + `npm ci`.
3. **Pushed branch ని rebase** – team history diverge. Local/private branches మాత్రమే rebase.
4. `git add .` **గుడ్డిగా** – unwanted files (logs, temp) కూడా. `git status` ముందు చూడు.
5. `git pull` **blindly** – surprise conflicts. `git fetch` + review + merge better.
6. `reset --hard` **uncommitted work మీద** – గోన్. ముందు `git stash`.
7. **చెడు commit messages** – `"fix"`, `"changes"`. `type: what and why`.
8. `git push --force` – team work overwrite. ఎప్పుడూ `--force-with-lease`.
9. **పెద్ద long-lived branches** – merge hell. Short-lived + తరచుగా merge.
10. **main కి direct push** – review లేదు, broken code. PR workflow + protected main.

🎯 SSE గా చివరి mindset

- **Terminal ని భయపడకు** – ప్రతి command reversible అనుకో (Git లో reflog, files కి backup). ప్రయోగించు, నేర్చుకో.
- `man` **+** `--help` **నీ friends** – flag గుర్తు లేకపోతే Google కంటే వేగం.
- **Automate repetitive work** – 3వ సారి అదే commands type చేస్తే, script రాయి.
- **Git లో దాదాపు ఏదీ నిజంగా పోదు** – panic అవ్వకు, `git reflog` + `git status` చూడు.
- **Production లో జాగ్రత్త** – `rm -rf`, `reset --hard`, `force-push` ముందు రెండుసార్లు ఆలోచించు. "measure twice, cut once".

Key Points

- Linux: **kernel/distro, permissions (755/644/600), processes (kill/ps/lsof), pipes+grep+awk, ssh/rsync, env/PATH.**
- Shell: **shebang, quoting `"$var"`, `if/for/while`, `$1/$@/$?`, `set -euo pipefail`, exit codes.**
- Git: **3 areas, branch/merge/rebase, conflicts, reset/revert/stash/reflog, PR workflow.**
- ఈ tools రోజూ terminal లో practice చెయ్ – reading కంటే typing తో muscle memory వస్తుంది.

Interview దృష్టి (final meta)

Q: ఒక Node app production లో crash అయ్యింది – ఎలా debug చేస్తావ్? (అన్ని skills కలిపి) A: (1) `ssh user@server` తో server కి login. (2) `pm2 list` / `systemctl status app` – app state చూస్తా. (3) `pm2 logs` / `journalctl -u app -f` / `tail -f /var/log/app.log | grep ERROR` – errors వెతుకుతా. (4) `top` / `htop` – CPU/RAM spike ఉందా. (5) `df -h` – disk full అయిందా. (6) `ss -tulpn | grep 3000` – port listening ఉందా. (7) root cause దొరికితే – code fix, `git` తో commit + PR, deploy. ఇది Linux + Git + scripting అన్నీ కలిపిన real SSE workflow – ఈ guide మొత్తం ఇందుకే.

> **ముగింపు:** ఈ 17 topics Linux, Shell scripting, Git ని absolute basics నుండి SSE-level depth వరకు cover చేశాయి – terminal భయం నుండి, server మీద confident గా deploy చేసి, logs debug చేసి, Git conflicts resolve చేసి, పోయిన commits recover చేసే స్థాయి వరకు. ప్రతి command కి real-life analogy, actual output, gotchas, interview angle, మరియు MERN workflow తో link ఇచ్చాం.
>
> ఇప్పుడు నీవు "self-taught developer" నుండి "అన్ని daily tools confident గా వాడే engineer" గా మారావు. ఈ tools ని రోజూ terminal లో type చేస్తూ practice చెయ్ – చదవడం కంటే చేయడం ద్వారానే ఇవి శాశ్వతంగా గుర్తుంటాయి.
>
> **"ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు" – ఇప్పుడు Linux, Shell, Git నీకు జీవితాంతం గుర్తుంటాయి.** 🚀